Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Prieskumník sémantiky logiky prvého rádu

Bakalárska práca

2018 Milan Cifra

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Prieskumník sémantiky logiky prvého rádu

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Aplikovaná informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Mgr. Ján Kľuka, PhD.

Bratislava 2018 Milan Cifra

37130592

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Milan Cifra
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: aplikovaná informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Prieskumník sémantiky logiky prvého rádu
First-order logic semantics explorer

Anotácia: Sémantika formúl a termov sa v klasickej matematickej logike uchopuje pomocou štruktúr a Tarského definície pravdivosti [1]. Osvojenie si tohto prístupu pri štúdiu základov logiky je z viacerých dôvodov náročné (abstraktnosť, rozmanitosť štruktúr, problém oddelenia syntaxe a sémantiky), a to až do takej miery, že niektorí autori preferujú herbrandovský prístup k sémantike [2]. Možnou cestou k pochopeniu sú cvičenia zamerané na rozhodovanie platnosti formúl v štruktúrach, hľadanie štruktúr (ne)spĺňajúcich dané formuly, hľadanie formúl splnených iba v niektorých štruktúrach a pod. V prípade konečných štruktúr sa správnosť riešenia takýchto úloh dá ľahko strojovo kontrolovať, a tým poskytnúť študentom rýchlu spätnú väzbu. Nie je nám však známy nástroj poskytujúci interaktívne vyhodnocovanie riešení a dostatočné možnosti zadávať rozličné úlohy.

Cieľ: Cieľom práce je navrhnúť, implementovať a otestovať prototyp interaktívneho webového nástroja – prieskumníka sémantiky logiky prvého rádu. Umožní definovanie prvorádového jazyka, konkrétnej konečnej štruktúry pre tento jazyk a kontrolu splnenia formúl. Prieskumník by mal slúžiť na riešenie úloh, ale aj na ich tvorbu. Vykonávať sa bude na strane klienta (t.j., vo webovom prehliadači), bude implementovaný vo vhodnom frameworku (React, AngularJS, elm, …).

Literatúra:
[1] Barwise, J. (ed.) (1982): Handbook of Mathematical Logic. North Holland.
[2] Genesereth, M., Kao, E. (2015): The Herbrand Manifesto. Thinking Inside the Box. In: Bassiliades N., Gottlob G., Sadri F., Paschke A., Roman D. +(eds) Rule Technologies: Foundations, Tools, and Applications. RuleML 2015. Lecture Notes in Computer Science, vol 9202. Springer.

Kľúčové slová: logika prvého rádu, splnenie v konečnej štruktúre, webová aplikácia na strane klienta

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: prof. Ing. Igor Farkaš, Dr.

Dátum zadania: 12.09.2017

Dátum schválenia: 16.10.2017 doc. RNDr. Damas Gruska, PhD.

37130592

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

garant študijného programu

.....

študent

.....

vedúci práce

Poďakovanie: Ďakujem školiteľovi práce, Mgr. Jánovi Kľukovi, PhD., za cenné rady, trpezlivosť, ochotu, odborné usmerňovanie a motiváciu pri písaní tejto práce. Taktiež chcem poďakovať študentom predmetu Matematika (4) - logika pre informatikov v letnom semestri akademického roka 2017/2018 ktorí testovali moju aplikáciu a doc. RNDr. Zuzane Kubincovej, PhD., Mgr. Dane Šuníkovej a Bc. Ivete Csicsolovej, ktorí mi pomohli pri zostavovaní dotazníka.

Abstrakt

Práca sa zaoberá tvorbou webovej aplikácie, ktorá ponúka študentom precvičovanie štruktúr logiky prvého rádu. V aplikácii si užívateľ môže definovať ľubovoľnú konečnú štruktúru, jazyk a výrazy (formuly a termy). Výrazy aplikácia vyhodnocuje na základe definovanej štruktúry. Taktiež kontroluje syntax výrazov, čím vynucuje ich správny zápis. Aplikácia môže byť v dvoch módoch, ktoré si používateľ prepína - v študentskom a učiteľskom, ktorý umožňuje zamykanie výrazov a častí definície štruktúry. Vytvorenú štruktúru a výrazy je možné exportovať do textového súboru, a ten sa dá naspäť importovať. Aplikácia je naprogramovaná pomocou knižníc React a Redux. Testovali ju študenti predmetu Matematika (4) - Logika pre informatikov v letnom semestri akademického roka 2017/2018.

Kľúčové slová: logika prvého rádu, splnenie v konečnej štruktúre, webová aplikácia na strane klienta

Abstract

The aim of this work was to develop a web application, which would allow students to learn about structures in first-order-logic. Users can define any finite structures, languages and expressions (formulas and terms). Application explores the structure through evaluating the expressions based on the defined structure. Students are required to write expressions using the proper syntax. Users can switch between two modes - student's and teacher's mode. Teacher's mode allows to lock expressions and parts of the structure's definition. Users can export and import the structures in text file format. The application is implemented using the React and Redux libraries. It was tested by students of course Matematika (4) - Logika pre informatikov in the summer term of academic year 2017/2018.

Keywords: first-order logic, satisfaction in finite structure, client-side web application

Obsah

Úvod **1**

1 Východiská práce **3**

1.1 Logika prvého rádu . . . 3

1.1.1 Jazyk . . . 3

1.1.2 Syntax . . . 4

1.1.3 Sémantika . . . 5

1.2 Použité technológie . . . 6

1.2.1 JavaScript, HTML, CSS, Bootstrap, npm . . . 6

1.2.2 React . . . 6

1.2.2.1 Elementy a komponenty . . . 6

1.2.2.2 Stav a životný cyklus komponentu . . . 8

1.2.2.3 Udalosti . . . 10

1.2.3 Redux . . . 10

1.2.4 PEG.js . . . 11

1.3 Podobné práce . . . 11

1.3.1 Introduction to Logic . . . 11

1.3.2 Tarski's World . . . 14

1.3.3 Logický pracovný stôl . . . 15

1.3.4 Výukový program demonštrujúci matematický princíp . . . 15

2 Analýza problému **16**

2.1 Účel aplikácie . . . 16

2.2 Požadované funkcie aplikácie . . . 17

2.3 Diagram prípadov použitia . . . 19

2.4 Ostatné požiadavky . . . 19

2.5 Analýza aplikačnej domény . . . 20

3 Návrh **21**

3.1 Dátový model . . . 21

3.2 Návrh stavu aplikácie . . . 22

3.3 Syntax textových vstupov 23
3.3.1 Definovanie jazyka 23
3.3.2 Definovanie štruktúry 24
3.3.3 Výrazy 24
3.4 Návrh komponentov používateľského rozhrania 26
3.5 Import a export 28

4 Implementácia 29
4.1 Implementačný jazyk 29
4.2 Štruktúra aplikácie 29
4.3 Štruktúra stavu 30
4.4 Komponenty 32
4.5 Syntaktická analýza vstupov 32
4.6 Akcie 33
4.7 Reducery a synchronizácia komponentov 34
4.8 Import a export 34
4.9 Zdrojové kódy a nasadenie 35

5 Testovanie 36
5.1 Cieľ testovania 36
5.2 Priebeh testovania 36
5.3 Výsledky testovania 36
5.4 Implementované vylepšenia 39

Záver 41

Príloha A 42

Príloha B 48

Príloha C 48

Úvod

Žijeme v modernej dobe s modernými spôsobami vyučovania. Je čím ďalej tým viac webových nástrojov a portálov, ktoré sa snažia uľahčiť, alebo zjednodušiť vyučovanie na školách. Študenti sú viac motivovaní, ak dostanú okamžite spätnú väzbu na nejakú úlohu, ktorú riešili. Výhoda takýchto webových nástrojov je aj fakt, že sú prakticky dostupné vždy a všade. Tieto webové nástroje sú buď prístupné iba po prihlásení, alebo také, kde je nástroj priamo otvorený pre všetkých užívateľom. Preto som sa rozhodol vytvoriť webový nástroj s názvom Prieskumník štruktúr, ktorý bude slúžiť ako prostriedok na precvičovanie logiky prvého rádu. Bude to otvorený nástroj, ktorý bude dostupný každému, bez nutnosti registrácie, resp. prihlásenia.

Cieľom práce je vytvoriť a otestovať online webový nástroj na tvorbu a riešenie úloh týkajúcich sa logiky prvého rádu. Bude slúžiť ako pomôcka ako pre študentov, tak aj pre učiteľov ktorý študujú, resp. učia túto problematiku. Používateľ si tu bude môcť definovať ľubovoľnú konečnú štruktúru a k nej jazyk. Túto štruktúru aplikácia umožní skúmať tak, že si používateľ bude môcť definovať ľubovoľné výrazy (formuly alebo termy), vyhodnotiť ich a tak zistiť, či štruktúra spĺňa formulu, resp. akú hodnotu má term. Samotné vyhodnocovanie výrazov bude prebiehať kvízovou formou. Aplikácia nazobrazí splnenie formuly či hodnotu termu, iba overí používateľom zvolenú možnosť. Od toho si sľubujeme väčšiu motiváciu k uvažovaniu a pochopeniu problematiky. Aplikácia bude od študenta vyžadovať zapisovať výrazy v správnej syntaxi, inak ich nebude možné vyhodnotiť. Aplikácia sa bude snažiť držať presných pomenovaní a znakov, ktoré definuje logika prvého rádu. Pre študentov je niekedy velký problém správne a dostatočne pochopiť pojmy ako sú štruktúra, term, jazyk, symboly jazyka a iné.

Dôvodom a hlavnou motiváciou vypracovania tejto záverečnej práce je fakt, že nám nie je známy žiadny iný podobný nástroj, ktorý by podobým spôsobom ponúkal možnosť precvičovania logiky prvého rádu. Jediným spôsobom, akým sa dali riešiť takéto úlohy bolo na papier, čo nebolo tak praktické, pretože študenti chcú poznať výsledok riešenia okmažite. Je veľmi ťažké sa naučiť správne zapisovať výrazy logiky, keď si to študent nemá ako overiť. Ďalším dôvodom vytvoriť takýto nástroj bol, aby študenti mali k dispozícii nástroj, vďaka ktorému si budú môcť precvičovať aj zadania, ktoré si sami vymyslia.

V prvej kapitole rozoberám hlavnú problematiku práce – logiku prvého rádu, použité technológie a podobné, existujúce práce. V druhej kapitole analyzujem problém, kde je

popísaný účel aplikácie a detailne formulované požiadavky. Tretia kapitola detailne rozoberá návrh aplikácie. Čitateľ sa v nej dozvie, akým jazykom, resp. knižnicami budem aplikáciu implementovať, grafický návrh aplikácie, alebo ako sa bude realizovať samotné definovanie jazyka, štruktúry a výrazov. Štvrtá kapitola je popis implementácie aplikácie. Posledná, piata kapitola ponúka čitateľovi výsledky testovania, ktoré bolo uskutočnené priamo na študentoch študujúcich logiku prvého rádu v akademickom roku vypracovávania tejto bakalárskej práce.

Kapitola 1

Východiská práce

V tejto kapitole zhrniem hlavnú problematiku práce a princípy fungovania prostriedkov ktoré som využil. Opíšem princípy logiky prvého rádu, ktoré sú nevyhnutné na pochopenie práce. Ukážem, aké prostriedky a technológie boli použité na vývoj aplikácie. Na záver v krátkosti analyzujem existujúce práce a aplikácie, ktoré ma určitým spôsobom inšpirovali pri tvorbe.

1.1 Logika prvého rádu

Logika prvého rádu je formálny systém používaný v matematike a informatike. Od jednoduchej výrokovej logiky sa odlišuje tým, že pridáva predikáty a kvantifikátory. Nasledujúce definície v tejto kapitole vychádzajú z prednášok [6] a Švejdarovej učebnice [11].

1.1.1 Jazyk

Jazyk $\mathcal{L}$ v prvorádovej logike je formálny jazyk ktorý definuje množinu konkrétnych symbolov, ktoré definujú syntax a sémantiku logiky. Jazyk obsahuje logické symboly, symboly indivíduových premenných, mimologické symboly a pomocné symboly.

Medzi *logické symboly* patria logické spojky $(\vee, \wedge, \rightarrow, \neg)$, symbol rovnosti $(\doteq)$ a kvantifikátory $(\forall, \exists)$.

Symboly indivíduových premenných sú symboly z nejakej nekonečnej spočítateľnej množiny $\mathcal{V}_{\mathcal{L}}$ a predstavujú objekty, ktoré nie sú konkrétne určené. Množina symbolov premenných môže byť definovaná napríklad takto:

$$\mathcal{V}_{\mathcal{L}} = \{\mathtt{x}, \mathtt{y}, \mathtt{z}, ...\} \tag{1.1}$$

Medzi *mimologické symboly* patria *symboly konštánt* $\mathcal{C}_{\mathcal{L}}$, *funkčné symboly* $\mathcal{F}_{\mathcal{L}}$ a *predikátové symboly* $\mathcal{P}_{\mathcal{L}}$. Všetkým predikátovým a funkčným symbolom je priradená *arita*. Arita je kladné prirodzené číslo a predstavuje počet argumentov symbolu. Arita sa zapisuje ako

horný pravý index pri názve symbolu. Napríklad $\texttt{kúpil}^2$ je predikátový symbol s aritou 2.

Symbol konštanty z množiny $\mathcal{C}_\mathcal{L}$ jazyka $\mathcal{L}$ predstavuje konkrétny objekt, ktorý sa nedá zameniť alebo konkrétnu hodnotu. Dá sa povedať, že sú podobné ako vlastné mená v prirodzenom jazyku alebo konštanty v programovacom jazyku. Napríklad symbol konštanty `Dom123` predstavuje konkrétny dom s číslom 123 v nejakej obci a nedá sa zameniť zo žiadnym iným. Množina symbolov konštánt môže byť definovaná napríklad takto:

$$\mathcal{C}_\mathcal{L} = \{\texttt{Dom123}, \texttt{Frantisek}, \texttt{Velkost}\} \tag{1.2}$$

Funkčný symbol z množiny $\mathcal{F}_\mathcal{L}$ jazyka $\mathcal{L}$ predstavuje jednoznačne určený vzťah. Napríklad funkčný symbol `cena(Produkt007)` predstavuje cenu produktu 007 v nejakom obchode. Produkt bežne máva iba jednu cenu, takže `Produkt007` má nejakú *jednoznačne* určenú cenu. Podobne, rozdiel dvoch cien $(-)$ je jednoznačne určený. Množina funkčných symbolov môže byť definovaná napríklad takto:

$$\mathcal{F}_\mathcal{L} = \{\texttt{cena}^1, \texttt{-}^2\} \tag{1.3}$$

Predikátový symbol z množiny $\mathcal{P}_\mathcal{L}$ jazyka $\mathcal{L}$ predstavuje určitú vlastnosť (napr. byť chlapcom) alebo vzťahy (napr. niekto niečo kúpil). Množina predikátových symbolov môže byť definovaná napríklad takto:

$$\mathcal{P}_\mathcal{L} = \{\texttt{kúpil}^2, \texttt{chlapec}^1\} \tag{1.4}$$

Množiny $\mathcal{V}_\mathcal{L}, \mathcal{P}_\mathcal{L}, \mathcal{F}_\mathcal{L}, \mathcal{C}_\mathcal{L}$ sú navzájom disjunktné. Medzi *pomocné symboly* patria `'('`, `')'` a `','`.

1.1.2 Syntax

Termy jazyka $\mathcal{L}$ pomenúvajú objekty. Základnými termami sú symboly konštánt a premenných. Ak t_1 až t_n sú termy a f je funkčný symbol s aritou n, tak aj $f(t_1, \ldots, t_n)$ je term. V jazyku $\mathcal{L}$ sú napríklad tieto postupnosti symbolov termami:

- $\texttt{x}, \texttt{y}, \texttt{z}$ (nekonkrétne objekty)
- $\texttt{Frantisek}, \texttt{Dom123}, \texttt{Velkost}$ (konkrétne objekty)
- $\texttt{cena(x)}, \texttt{cena(Dom123)}, \texttt{-(x,y)}$ (objekty, ktoré sú v jednoznačnom vzťahu s inými objektmi)

Atomické formuly jazyka $\mathcal{L}$ tvoria množinu $\mathcal{A}_\mathcal{L}$. Medzi atomické formuly patria rovnostné atómy a predikátové atómy. Ak t_1 až t_n sú termy, tak postupnosť symbolov $t_1 \doteq t_2$ sa nazýva *rovnostný atóm* jazyka $\mathcal{L}$. Rovnostné atómy vyjadrujú, že dva termy ukazujú na

ten istý objekt. Ak t_1 až t_n sú termy a P je predikátový symbol s aritou n, tak postupnosť symbolov $P(t_1, \ldots, t_n)$ sa nazýva *predikátový atóm* jazyka $\mathcal{L}$.

Množina *formúl* jazyka $\mathcal{L}$ je najmenšia množina induktívne definovaná nasledovne:

- všetky atomické formuly sú formulami,
- ak sú α a β formuly, tak aj $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, $\neg\alpha$ sú formulami,
- ak x je indivíduová premenná a α je formula, tak aj $\forall x\alpha$ a $\exists x\alpha$ sú formulami.

1.1.3 Sémantika

Hodnotu formuly alebo termu jazyka $\mathcal{L}$ určuje *štruktúra*. Štruktúrou pre jazyk $\mathcal{L}$ je dvojica $\mathcal{M} = (M, i)$, kde M je doména štruktúry a i je interpretačná funkcia štruktúry.

Doména štruktúry je množina objektov, ktoré môžu byť pomenované termami.

Interpretačná funkcia je zobrazenie, ktoré každému symbolu konštanty c jazyka $\mathcal{L}$ priraďuje prvok $i(c) \in M$, každému funkčnému symbolu f jazyka $\mathcal{L}$ s aritou n priraďuje funkciu $i(f)\colon M^n \rightarrow M$ a každému predikátovému symbolu P jazyka $\mathcal{L}$ s aritou n priraďuje množinu $i(P) \subseteq M^n$.

Ohodnotenie (indivíduových) premenných jazyka $\mathcal{L}$ je ľubovoľná funkcia $e\colon \mathcal{V}_{\mathcal{L}} \rightarrow M$, ktorá každej premennej priraďuje prvok z domény.

Hodnotou termu t v štruktúre $\mathcal{M}$ pri ohodnotení premenných e je prvok $t^{\mathcal{M}}[e]$ z M určený nasledovne:

- $x^{\mathcal{M}}[e] = e(x)$, ak x je premenná,
- $a^{\mathcal{M}}[e] = i(a)$, ak a je konštanta,
- $(f(t_1, \ldots, t_n))^{\mathcal{M}}[e] = i(f)(t_1^{\mathcal{M}}[e], \ldots, t_n^{\mathcal{M}}[e])$, ak $t_1, \ldots, t_n$ sú termy a f je funkčný symbol s aritou n.

Relácia *štruktúra $\mathcal{M}$ spĺňa formulu A pri ohodnotení e* (skrátene $\mathcal{M} \models A[e]$) má nasledovnú induktívnu definíciu:

- $\mathcal{M} \models t_1 \doteq t_2[e]$ vtt $t_1^{\mathcal{M}}[e] = t_2^{\mathcal{M}}[e]$,
- $\mathcal{M} \models P(t_1, \ldots, t_n)[e]$ vtt $(t_1^{\mathcal{M}}[e], \ldots, t_n^{\mathcal{M}}[e]) \in i(P)$,
- $\mathcal{M} \models \neg A[e]$ vtt $\mathcal{M} \not\models A[e]$,
- $\mathcal{M} \models (A \wedge B)[e]$ vtt $\mathcal{M} \models A[e]$ a zároveň $\mathcal{M} \models B[e]$,
- $\mathcal{M} \models (A \vee B)[e]$ vtt $\mathcal{M} \models A[e]$ alebo $\mathcal{M} \models B[e]$,
- $\mathcal{M} \models (A \rightarrow B)[e]$ vtt $\mathcal{M} \not\models A[e]$ alebo $\mathcal{M} \models B[e]$,

- $\mathcal{M} \models \exists x A[e]$ vtt pre nejaký prvok $m \in M$ máme $\mathcal{M} \models A[e(x/m)]$,
- $\mathcal{M} \models \forall x A[e]$ vtt pre každý prvok $m \in M$ máme $\mathcal{M} \models A[e(x/m)]$,

pre všetky arity $n > 0$, všetky predikátové symboly P s aritou n, všetky termy $t_1, t_2, \ldots, t_n$, všeky premenné x a všetky formuly A, B.

Formula A je *splniteľná* vtt aspoň jedna štruktúra $\mathcal{M}$ pre jazyk $\mathcal{L}$ spĺňa A pri aspoň jednom ohodnotení e.

1.2 Použité technológie

V tejto sekcii opisujem technológie, ktoré budú použité pri tvorbe aplikácie.

1.2.1 JavaScript, HTML, CSS, Bootstrap, npm

V práci využívam štandardné technológie na tvorbu webových aplikácií ako sú HTML, CSS a JavaScript. Na štýlovanie webu som použil CSS knižnicu Bootstrap verzie 3. Využívam kompilátor Gulp a jeho plugin `gulp-sass`, pomocou ktorého kompilujem štýl webu napísaný v Sass do CSS. Na spravovanie všetkých JavaScriptových knižníc využívam manažér balíkov npm.

1.2.2 React

React je JavaScriptová knižnica, ktorej hlavným cieľom je efektívna tvorba webových používateľských rozhraní, a to je dôvod využitia tejto knižnice v práci. Bola vytvorená spoločnosťou Facebook a komunitou vývojárov. Originálnym autorom je vývojár v spoločnosti Facebook, Jordan Walke. Prvá verzia React-u vyšla v marci v roku 2013 [2].

V tejto kapitole sa budem snažiť v krátkosti opísať architektúru React-u na príklade, kde budem vytvárať rozhranie na pridávanie textových reťazcov do zoznamu. Nechcem zachádzať do detailov, budem v krátkosti opisovať iba tie aspekty, ktoré využijem pri tvorbe aplikácie. Ukážky kódov sú zjednodušené, aby boli ľahšie čitateľné. Zdrojom k tejto sekcii je hlavná, online dokumentácia React-u [5].

1.2.2.1 Elementy a komponenty

React je vo všetkých smeroch Javascriptová knižnica. Nekombinujú sa v nej súbory v HTML a v Javascripte ako je to pri klasických MVC frameworkoch. V aplikáciách tvorených pomocou Reactu sa stránky nevytvárajú pomocou klasických dokumentov v HTML, kde sa píšu elementy HTML, ale namiesto toho sa vytvárajú Javascriptové *objekty* elementov HTML. Objekt elementu obsahuje typ, atribúty a potomkov, ktoré by mal štan-

dardný HTML element. Napríklad element typu `button` môže obsahovať atribúty `id`, `class`, `style` a iné.

Element v Reacte je objektová reprezentácia elementu HTML. Každý element má definovaný typ, atribúty a zoznam potomkov, ktoré sú tiež elementy. Element sa v Reacte vytvára pomocou funkcie `createElement` (výpis 1.1).

```
const button = React.createElement (
    'button',
    {id: 'add-btn'},
    'Pridaj'
);
```

Výpis 1.1: Vytvorenie elementu

Prvý argument funkcie je typ elementu. Môže to byť reťazec alebo objekt už existujúceho elementu. Druhý argument (`{id: 'add-btn'}`) je objekt atribútov elementu. Tretí argument je zoznam potomkov elementu. V tomto príklade je len jeden potomok – textový reťazec. Funkcia `createElement` vráti objektovú reprezentáciu elementu (výpis 1.2).

```
{
    type: 'button',
    props: {
        id: 'add-btn',
        children: 'Pridaj'
    }
}
```

Výpis 1.2: Objektová reprezentácia vytvoreného elementu

Takto sa v Reacte vytvára strom elementov, ktoré sa nakoniec vykreslia ako klasické elementy HTML. Vykreslenie tohto stromu zabezpečuje funkcia `ReactDOM.render` (výpis 1.3). Stačí jej povedať aký element vykresliť a kde – element HTML v dokumente v HTML. Štandardne každá Reactová aplikácia obsahuje iba jeden dokument HTML – `index.html`. Obsahuje nevyhnutné elementy akými sú `<html>`, `<head>` a `<body>`. Element `<body>` štandardne obsahuje iba jeden element `<div id='root'>`, do ktorého React vykreslí spomínaný strom elementov aplikácie.

```
ReactDOM.render(button, document.getElementById('root'));
```

Výpis 1.3: Vykreslenie elementu do DOM

Komponent je o niečo zložitejší element. Komponent je väčšinou definovaný ako trieda, ktorá má konštruktor a metódu `render()`. Okrem toho môže mať ľubovoľný počet ďalších metód. Konštruktor prijíma jeden parameter `props`. Je to štandardný objekt, ktorý obsahuje atribúty ktoré sa zadajú pri vytváraní komponentu. Základným rozdielom medzi elementom a komponentom je, že komponent príjima atribúty, a na základe nich vracia elementy. V našom príklade budú dva komponenty – `App` a `InputValue`. Komponent `App`

bude obsahovať element `ul` (zoznam, v ktorom budú užívateľom pridané textové reťazce), komponent `InputValue` a element `button` na pridávanie do zoznamu.

Vytváranie elementov pomocou funkcie `createElement` je príliš nepohodlné, preto druhým a jednoduchším spôsobom vytvárania elementov a komponentov je pomocou JSX, čo je syntaktické rozšírenie JavaScriptu. Pomocou JSX sa dajú elementy vytvárať pomocou klasických HTML tagov a vytvárať tak prehľadnejší a ľahšie pochopiteľnejší kód. Element `button` ktorý sme vytvárali vo výpise 1.1 sa dá prepísať do syntaxe JSX spôsobom znázorneným vo výpise 1.4. Takýmto spôsobom sa dajú vytvárať aj komponenty (výpis 1.5).

```
<button id='add-btn'>Pridaj</button>
```

Výpis 1.4: Vytvorenie elementu pomocou JSX

```
<App>
 <ul> </ul>
 <InputValue>
 <button id='add-btn'>Pridaj</button>
</App>
```

Výpis 1.5: Vytvorenie komponentu pomocou JSX

1.2.2.2 Stav a životný cyklus komponentu

Ako som spomínal v predchádzajúcej sekcii, každý komponent obsahuje metódu `render()` ktorá vykresľuje komponent užívateľovi. Každý komponent môže obshovať *stav*. Stav komponentu je jednoduchý objekt, do ktorého je možné uložiť ľubovoľné množstvo dát. Ak sa obsah stavu zmení, Ract zavolá metódu `render()`, a komponent sa nanovo prekreslí. Objekt stavu má zaužívané meno `state` ale môže mať ľubovoľné meno. V stave komponentu by mali byť iba tie údaje, ktoré ovplyvňujú vykresľovanie komponentu.

Komponent `App` v našom príklade sa má prekresliť vtedy, keď používateľ pridá prvok do zoznamu. To znamená, že keď klikne na element `button`, zoberie sa hodnota z komponentu `TextValue` a táto hodnota sa pripojí k elementu `ul`. Preto potrebujeme, aby v stave tohto komponentu bolo pole `items`, ktoré bude obsahovať tieto používateľom pridané hodnoty.

Počiatočný objekt stavu sa definuje v konštruktore komponentu (výpis 1.6). Komponent `App` bude mať v stave prázdne pole do ktorého sa budú neskôr pridávať textové reťazce ktoré zadá používateľ.

```
class App extends React.Component {
 constructor(props) {
  super(props);
  this.state = {
   items: []
  };
```

```
 }
 ...
}
```

Výpis 1.6: Definovanie stavu v konštruktore komponentu

Priame priradenie objektu k premennej `this.state` sa odporúča iba v konštruktore. Mimo konštruktora by sa mal stav definovať pomocou funkcie `setState(newState)`, ktorá zabezpečí, aby sa objekt prekreslil. Táto funkcia dostáva ako argument zmenený objekt stavu. Výpis 1.7 ukazuje pridanie prvku do stavu. Po vykonaní tejto funkcie bude element prekreslený a v elemente `ul` pribudne `<li>item</li>`.

```
addItem(item) {
 newItems = this.state.items.push(item);
 this.setState({
  items: newItems
 });
}
```

Výpis 1.7: Zmena stavu komponentu pomocou funkcie `setState()`

Nakoniec je potrebné definovať, ako sa bude komponent vykresľovať ako je znázornené vo výpise 1.8. Funkcia `render()` vracia objekt elementu ktorý je napísaný v JSX syntaxi. Najdôležitejšia časť je na riadkoch 8 – 10. Takýmto spôsobom sa dá vypísať každý prvok poľa `items` do HTML elementu `li`.

```
class App extends React.Component {
 ...
 render() {
  return (
   <div class="list">
    <ul>
     {
      this.state.items.map((value, index) =>
       <li key={value}>{value}</li>
      )
     }
    </ul>
    <TextInput />
    <button id='add-btn'>Pridaj</button>
   </div>
  )
 }
 ...
}
```

Výpis 1.8: Vykresľovanie komponentu `App`

1.2.2.3 Udalosti

Najdôležitejšou súčasťou Reactu je reagovanie na rôzne udalosti od používateľa. Každý komponent môže mať definované, akú funkciu má vykonať pri vyvolaní nejakej udalosti. Okrem štandardných udalostí ktoré definuje React (napr. `onChange()`), je možné vytvárať aj vlastné udalosti. Napríklad ak chceme, aby sa vykonala nejaká funkcia pri zmene stavu nejakého komponentu, túto funkciu mu dáme medzi atribúty.

V našom príklade máme komponent `TextInput`, ktorý má stav aktuálnej hodnoty elementu `input`. Chceme, aby sa pri zmene tejto hodnoty, teda zmene stavu tohoto komponentu, dozvedel túto hodnotu jeho rodič, teda komponent `App`. Dá sa to dosiahnuť práve pomocou jednoduchej udalosti. Upravíme vytvorenie komponentu v komponente `App` tak, že mu túto udalosť pridáme medzi atribúty (výpis 1.9). Funkcia `updateNewValue()` iba aktualizuje stav komponentu `App` zmenením tejto hodnoty.

```
<InputText onValueChange={this.updateNewValue()} />
```

Výpis 1.9: Vykresľovanie komponentu `TextInput` - pridaná udalosť `onValueChange`

Takýmto elegantným spôsobom sa dajú v Reacte informovať rodičovské komponenty a napríklad im tak poslať dáta, ktoré potrebujú. Dá sa takýmto spôsobom oznámiť vnoreným komponentom čo a kedy majú spraviť alebo vrátiť.

Štandardým elementom HTML sa dajú priradiť adekvátne štandardné udalosti. Jednou z nich ktoré využijeme v našom príklade je udalosť `onClick` na elemente `button` (výpis 1.10).

```
<button onClick={this.addItem()}>Pridaj</button>
```

Výpis 1.10: Vykresľovanie elementu `button` - pridaná udalosť `onClick`

Funkcia `addItem()` pridá tento prvok medzi prvky v stave, a tak sa prekreslí komponent `App` čím sa tento prvok zobrazí v prehliadači medzi ostatnými prvkami.

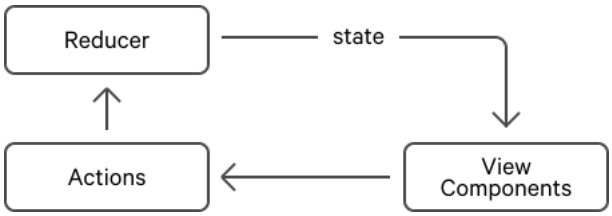
1.2.3 Redux

Redux [1] je JavaScriptová knižnica vytvorená na spravovanie stavu aplikácií bežiacich v internetovom prehliadači. Stav aplikácie je objekt, do ktorého si aplikácia ukladá dáta. Je to objekt, ktorý popisuje, v akom stave sa aplikácia nachádza. Do stavu je možné uložiť ľubovoľné množstvo dát (napr. hodnoty textových vstupov) a tie následne nejak spracovať. Výhodou je, že všetky tieto dáta sú centralizované a preto je spravovanie oveľa jednoduchšie.

Na obr. 1.1 je diagram znázorňujúci tok údajov v Reduxe. *Action* je klasický JavaScriptový objekt. Nesie informáciu, čo sa stalo (napr. používateľ klikol na tlačidlo). Môže mať ľubovoľné atribúty, avšak musí mať jeden povinne - `type`. Tento atribút, čo je ľubovoľný textový reťazec, jednoznačne identifikuje akciu. Akcia sa zavolá z nejakého Reactového

komponentu a zavolá sa reducer. *Reducer* je obyčajná funkcia, ktorá prijíma ako argumenty `action` (akciu) a `state` (aktuálny stav aplikácie). Reducer zmení aktuálny stav podľa akcie, ktorá sa má vykonať a vráti nový stav vďaka čomu sa používateľské rozhranie (Reactový komponent) prekreslí. Stav je nemeniteľný, čo znamená že reducer stav nemení pôvodný, ale vytvorí kópiu starého, ktorý zmení a vráti.

Na naše účely by postačoval iba React, avšak dôvod zakomponovania bol, že Redux má kód, ktorý mení stav aplikácie na jednom mieste a oddeľuje logiku spracovania stavu od samotného zobrazovania údajov používateľovi. Kód je vďaka tomu prehľadnejší a ľahšie rozšíriteľný.



Obr. 1.1: Tok údajov v Reduxe

1.2.4 PEG.js

PEG.js [7] je jednoduchý generátor syntaktických analyzátorov (parserov) s otvoreným zdrojovým kódom pre JavaScript. Pomocou jednoduchej syntaxe je možné vytvoriť mnoho pravidiel na parsovanie textového reťazca a tie následne skompilovať do JavaScriptu. Využijeme ho na parsovanie formúl a iných vstupov od užívateľa. Vybral som si ho najmä kvôli jeho jednoduchosti, rýchlosti a širokej komunite.

1.3 Podobné práce

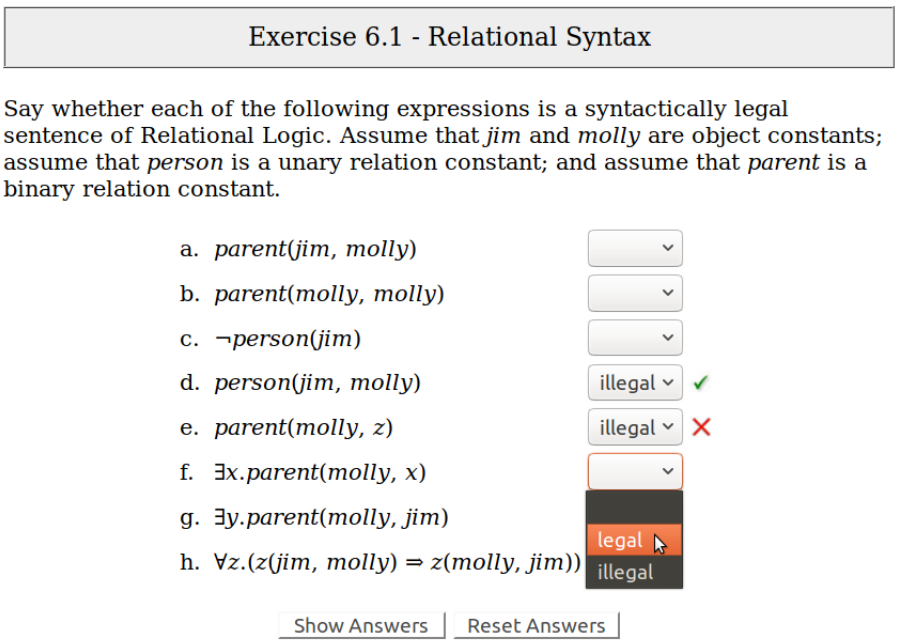
V tejto sekcii uvádzam tri práce, ktoré sú inšpiratívnym zdrojom, alebo sa tiež týkajú matematickej logiky.

1.3.1 Introduction to Logic

Webový nástroj od univerzity Stanford [4] je práca, ktorá je našou hlavnou inšpiráciou. Je to nástroj na online výučbu a precvičovanie matematickej logiky. Obsah je rozdelený do 12 kapitol, začínajúc základnou výrokovou logikou a končiac pokročilými dôkazmi a rezolvenciou. Každá kapitola začína krátkou prednáškou, po ktorej nasledujú rôzne interaktívne cvičenia. Naším cieľom je prevziať z tohto nástroja spôsoby, akým sú vedené tieto cvičenia, zamerať sa iba na logiku prvého rádu a rozšíriť funkcionalitu. Pre účely

tejto práce sú zaujímavé cvičenia týkajúce sa kontrolovania syntaxe formúl alebo hľadanie pravdivostnej hodnoty predikátu, aby formula skladajúca sa z tohto predikátu bola pravdivá.

Jedným z cvičení ktoré je pre nás zaujímavé, je precvičovanie syntaxe formúl, ako je zobrazené na obr. 1.2. V tomto cvičení je zoznam formúl, a pri každej je možnosť vybrať, či je formula zapísaná správne alebo nie. Po vybratí možnosti sa vedľa zobrazí, či je odpoveď správna. Tento nástroj neobsahuje cvičenie, kde by sa kombinovali všetky termy a formuly dokopy. Na obr. 1.2 sú formuly, ktoré obsahujú iba predikáty, konštanty a premenné. Cieľom je typ tohoto cvičenia rozšíriť na takú úroveň, aby si učiteľ vedel v administratívnej časti vytvoriť ľubovolný počet formúl a dať ich študentovi cvičiť.



Obr. 1.2: Cvičenie zamerané na precvičovanie syntaxe.

Ďalším typom cvičení je vyhodnocovanie predikátov, čo znázorňuje obr. 1.3. Je zadaná fixná relačná tabuľka, ktorá označuje, kedy je predikát splnený a študent musí podľa toho rozhodnúť, či je formula, ktorá tento predikát obsahuje, splnená alebo nie. Podobne ako pri cvičení syntaxe, aj tu sú na výber dve možnosti a zobrazenie výsledku. Nástroj obsahuje aj presne opačné cvičenie k tomuto, ktoré je na obr. 1.4. Tu sú fixne dané vety, a je potrebné „vyklikať“ tabuľku tak, aby všetky vety boli splnené.

Problémom tohto nástroja je v tom, že je určený pre verziu prvorádovej logiky, ktorej sémantika nie je daná štruktúrami. Používateľ si nevie definovať vlastnú, ľubovoľnú štruktúru a výrazy a tak zisťovať, aký vťahy sú medzi nimi. Neobsahuje editor výrazov, ale iba preddefinované výrazy. V tomto nástroji nie je možné skúmať splnenie formúl v štruktúre a zisťovať, aké zmeny nastanú po zmene štruktúry.

Exercise 6.4 - Relational Evaluation

This exercise concerns the interpersonal relations of a small sorority. There are just four members - Abby, Bess, Cody, and Dana; and there is just one type of binary relationship - likes. The following table below shows who likes whom. A check in a box of the table indicates that the girl named at the beginning of the row likes the girl named at the head of the column; the absence of a check means that she does not.

	Abby	**Bess**	**Cody**	**Dana**
Abby		✓		✓
Bess	✓		✓	
Cody		✓		✓
Dana	✓		✓	

The following sentences are constraints that characterize the possibilities. Your job here is to select a truth value for each constraint indicating whether that constraint is satisfied by the table shown above.

a. *likes*(*dana*,*cody*) true ✓
b. ¬*likes*(*abby*,*dana*) false ✓
c. *likes*(*bess*,*cody*) ∨ *likes*(*bess*,*dana*) false ✗
d. ∀*y*.(*likes*(*bess*,*y*) => *likes*(*abby*,*y*)) true ✗
e. ∀*y*.(*likes*(*y*,*cody*) ⇒ *likes*(*cody*,*y*)) true false
f. ∀*x*.¬*likes*(*x*,*x*)

Show Answers Reset Answers

Obr. 1.3: Hľadanie pravdivostnej hodnoty formuly pri danej fixnej tabuľke pravdivostných hodnôt predikátu `likes`.

Problem 7.2 - Boolean Models

Amy, Bob, Coe, and Dan are traveling to different places. One goes by train, one by car, one by plane, and one by ship. Amy hates flying. Bob rented his vehicle. Coe tends to get seasick. And Dan loves trains. Use the Boolean model technique and the following table to figure out which person, used which mode of transportation. Clicking on an empty cell in the table makes that cell true; clicking on a true cell makes it false; and clicking on a false cell makes its truth value unknown.

	train	car	plane	ship
Amy	0	0	0	1
Bob	0	1	0	0
Coe	0		1	1
Dan	1	0	0	0

The sentences in the table below capture the available information. As you change the world, the truth values of these sentences are recomputed and displayed in this table.

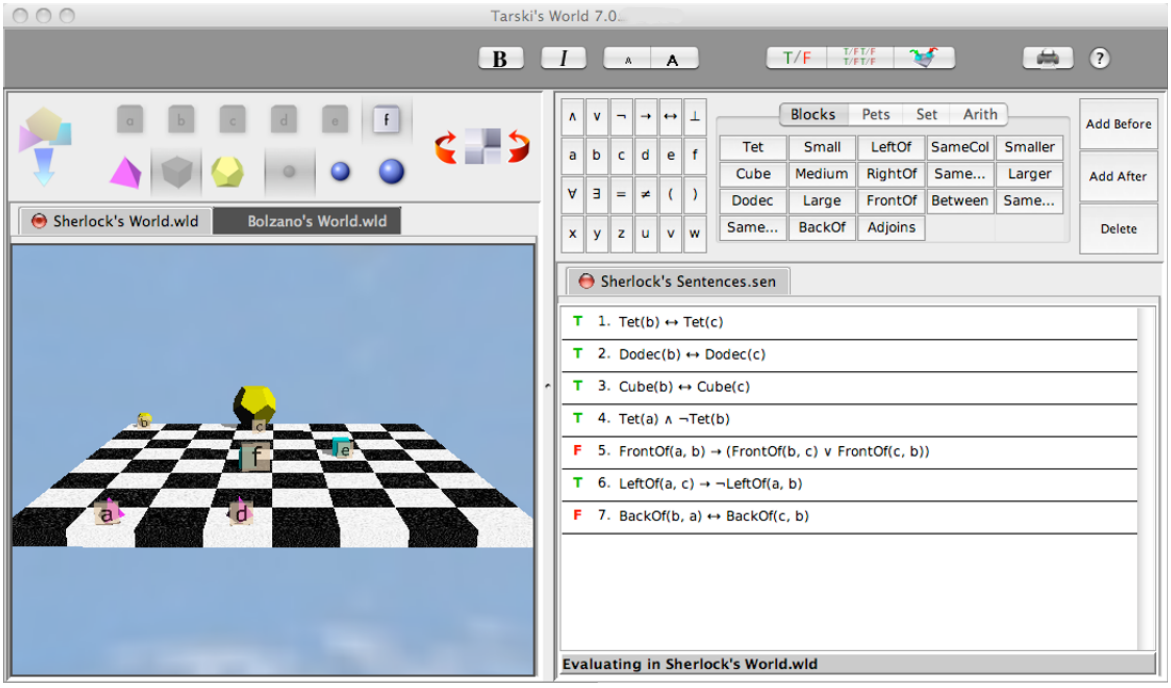
Sentence	Truth Value
Each person took a train or a car or a plane or a ship.	✓
No two people used the same method of transportation.	✗
Amy hates flying	✓
Bob rented his vehicle.	✓
Coe gets seasick.	✗
Dan loves trains.	✓

Obr. 1.4: Hľadanie kombinácie kedy sú všetky vety pravdivé.

1.3.2 Tarski's World

Tarski's World [8] je multi-platformová desktopová aplikácia zameraná na precvičovanie základov jazyka a sémantiky logiky prvého rádu (obr. 1.5). Ponúka používateľovi si vytvoriť vlastný 3D svet a popísať ho v jazyku prvorádovej logiky. Svet môže pozostávať z kolekcie telies (kocky, ihlany, gule, ...), ktoré môžu mať rôznu veľkosť. Tieto telesá sa ukladajú na šachovnicu v editore. Takýto svet zodpovedá konečnej štruktúre pre konkrétny prvorádový jazyk. V aplikácii sa dá zapísať ľubovoľná formula, ktorú aplikácia vyhodnocuje na základe vytvoreného sveta. S používateľom hrá hru, počas ktorej aplikácia na základe používateľovho úvodného rozhodnutia o splnení celej formuly vyvodzuje informácie o platnosti podformúl. Hra sa končí úspechom používateľa, ak dovedie aplikáciu k atomickej formule, ktorej odvodená pravdivostná hodnota skutočne zodpovedá situácii vo svete.

Podobnosťou práce je, že si v nej používateľ môže vytvoriť vlastnú štruktúru a skúmať ju vyhodnocovaním formúl. Navyše dokáže vyvodzovať informácie o platnosti podformúl. Nevýhodou je, že má obmedzený jazyk ktorým sa môžu pomenovať telesá.



Obr. 1.5: Aplikácia Tarski's World [8]

1.3.3 Logický pracovný stôl

Bakalárska práca s názvom *Logický pracovný stôl* [9] sa zaoberá výrokovou logikou. Výsledkom tejto práce je aplikácia, ktorá ponúka užívateľovi interaktívne pracovať s formulami výrokovej logiky, vie zistiť, či je formula splniteľná, nesplniteľná, alebo či je to tautológia. Podobne ako pri mojej práci, obsahuje vstup, kde sa dá zapísať ľubovoľná formula. Tento vstup sa následne sparsuje a napríklad zistiť, či je to tautológia alebo nie.Táto práca má, rovnako ako tá moja, implementované vyhodnocovanie výrazov, ale iba pre výrokovú logiku.

1.3.4 Výukový program demonštrujúci matematický princíp

Druhá bakalárska práca ktorá sa zaoberá metematickou logikou je *Výukový program demonštrujúci matematický princíp* [10]. Práca sa zaoberá tvorbou webovej aplikácie, kde si užívateľ precvičuje postupy matematických dôkazov. Konkrétne sa zaoberá dokazovaním rovnosti viet Boolovej algebry. Užívateľ si dôkaz konkrétnej pred-pripravenej vety vo forme rovnice môže vyskladať pomocou axióm.

Kapitola 2

Analýza problému

Táto kapitola upresňuje požiadavky, ktoré musí naša aplikácia spĺňať. Na začiatku v krátkosti opisujem, čo sa od aplikácie očakáva, resp. aký je jej účel. Ďalej konkrétne formulujem požadovanú funkcionalitu aplikácie. Uvádzam aj diagram prípadov použitia. Na záver formulujem ostatné požiadavky na aplikáciu, ktoré nepatria medzi funkčnú špecifikáciu.

2.1 Účel aplikácie

Aplikácia má umožniť študentom precvičovanie sémantiky logiky prvého rádu a učiteľom vytvoriť takéto cvičenia. V aplikácii sa bude musieť dať *definovať jazyk* a *štruktúra* logiky prvého rádu. Ďalej je požadované, aby sa v aplikácii dali *vyhodnocovať formuly* a *termy*. Používateľ napíše ľubovoľný výraz, a *vyberie* podľa vlastného uváženia, či formula spĺňa štruktúru, resp. akú hodnotu má term. Následne aplikácia *skontroluje*, či je jeho odpoveď správna.

Mala by obsahovať dva užívateľské módy – mód študenta a mód učiteľa. V *učiteľskom móde* má byť umožnené zamykať vstupy. To znamená, že akákoľvek definícia (napr. definícia domény štruktúry) sa bude dať „zamknúť“, a ďalej sa s touto definíciou nedá manipulovať až do jej odomknutia. Takýmto spôsobom sa budú dať vytvárať zadania na cvičenie pre študentov. Funkcie *študentského módu* budú vlastne podmnožinou funkcií toho učiteľského s tým rozdielom, že študent nebude môcť nijak manipulovať so zamknutými definíciami, vrátane zamykania a odomykania.

Vytvorené, resp. rozpracované zadanie by sa malo buď niekde *uložiť*, alebo *prevziať* do počítača používateľa. Naopak, aplikácia by mala zvládnuť aj *načítanie* takéhoto rozpracovaného zadania.

Pri akejkoľvek zmene ktorú vykoná používateľ musí aplikácia správne zareagovať a táto zmena sa musí prejaviť na tých správnych miestach.

2.2 Požadované funkcie aplikácie

V tejto sekcii uvádzam konkrétne funkcie, ktoré musí aplikácia spĺňať. Vznikali postupne na konzultáciach so školiteľom práce.

1) Editor jazyka

   a) Používateľ si bude môcť v tomto editore definovať mimologické symboly jazyka - symboly konštánt, predikátové symboly a funkčné symoly.

   b) Pri definovaní predikátových a funkčných symbolov je nutné definovať aj aritu. Arita má byť celé nezáporné číslo.

   c) Ak používateľ zadá ľubovoľný mimologický symbol s rovnakým menom viackrát, aplikácia ho na to upozorní a tento symbol bude ignorovať. Teda každý mimologický symbol jazyka bude v množine jazyka najviac jeden krát.

2) Editor štruktúry a ohodnotenia premenných

   a) Používateľ si bude môcť definovať množinu prvkov domény. Duplicitné prvky sa budú ignorovať a nie je potreba o tom informovať používateľa.

   b) Informovať používateľa, ak je množina domény prázdna. Platí, že táto množina nesmie byť prázdna.

   c) Používateľ si interpretáciu každého symbolu konštanty bude môcť vybrať zo zoznamu prvkov, ktoré budú patriť do domény štruktúry.

   d) Informovať používateľa, ak nie je definovaná interpretácia symbolu konštanty.

   e) Používateľ bude môcť definovať interpretáciu predikátových symbolov *textovým vstupom* alebo editačnou maticou. Ak interpretáciu definuje cez textový vstup, zadáva n-tice kde n je arita tohto symbolu, a všetky prvky musia patriť do domény štruktúry.

   f) Používateľ bude môcť definovať interpretáciu funkčných symbolov *textovým vstupom* alebo editačnou maticou. Ak ohodnotenie definuje cez textový vstup, zadáva $(n+1)$-tice, kde n je arita funkčného symbolu. Prvých n prvkov budú argumenty funkcie a posledný bude hodnota funkcie pre tieto argumenty. Všetky prvky musia patriť do domény štruktúry.

   g) Ak interpretácia funkčného symbolu priraďuje rovnakej n-tici argumentov viac rôznych hodnôt, aplikácia o tom informuje používateľa a tieto kolidujúce n-tice (kvôli nejednoznačnosti) bude ignorovať.

   h) Ak nie je funkčný symbol definovaný pre nejakú n-ticu argumentov, aplikácia o tom informuje používateľa, aby toto ohodnotenie definoval.

i) *Editačná matica* bude tabuľka, ktorá bude sekundárnou možnosťou editovania interpretácie predikátových a funkčných symbolov. Bude sa dať zobraziť, len ak má symbol aritu 1 alebo 2. Bude mať toľko stĺpcov, koľko prvkov má doména štruktúry, a názov každého stĺpca bude prvok domény. Ak je arita symbolu 1, tak bude mať práve jeden riadok, a ak je arita symbolu 2, tak budú označené aj riadky prvkami domény. V každej bunke bude buď zaškrtávacie políčko (pri predikátovom symbole) alebo menu s prvkami domény (pri funkčnom symbole).

j) Používateľ bude môcť definovať ohodnotenie indivíduových premenných cez textový vstup – podobne ako pri definovaní interpretácie predikátových symbolov, s tým rozdielom, že bude potrebné zapísať dvojice, kde prvý prvok bude názov premennej a druhý prvok domény – musí byť v doméne. Platí, že premenná je každý symbol, ktorý nie je v jazyku. Ak používateľ zadá symbol, ktorý už je v jazyku, aplikácia ho o tom informuje a toto ohodnotenie ignoruje.

3) Vyhodnocovanie výrazov

a) Používateľ bude môcť zadať ľubovoľný počet výrazov a aj ich zmazať.

b) Aplikácia musí kontrolovať správnosť syntaxe výrazu a upovedomiť o prípadných chybách používateľa.

c) Logické znaky vo výraze (spojky, kvantifikátory, rovnosť a nerovnosť) sa majú dať zapísať viacerými spôsobmi – ako LaTeXové príkazy, ascii art alebo priamym vložením unikódového znaku.

d) Pri každej formule musí mať používateľ možnosť vybrať odpoveď, či štruktúra spĺňa formulu alebo nie. Pri každom terme majú byť v možnostiach odpovede všetky prvky domény štruktúry.

e) Po výbere odpovede aplikácia skontroluje jej správnosť a používateľovi vypíše správu o správnosti.

4) Synchronizácia komponentov

a) Po pridaní symbolu do jazyka sa v editore štruktúry vytvorí vstup na zadanie interpretácie tohto symbolu. Ak sa tento symbol z jazyka zmaže, zmaže sa aj tento vstup.

b) Po zmene symbolov jazyka je potrebné znova vyhodnotiť všetky výrazy definované používateľom.

c) Po zmene interpretácie symbolov jazyka alebo ohodnotenia premenných sa musia znova vyhodnotiť všetky výrazy definované používateľom.

d) Po zmene domény štruktúry je potrebné skontrolovať interpretácie všetkých mimologických symbolov jazyka a ohodnotenia indivíduových premenných.

5) Výber používateľského módu a zamykanie vstupov

   a) Používateľ bude môcť vybrať dva editovacie módy – učiteľský a študentský.

   b) Používateľ v *učiteľskom móde* bude môcť vstup zamknúť a tak s ním už ďalej nebude môcť manipulovať až do jeho odomknutia.

   c) Používateľ v *študentskom móde* nebude môcť vstupy zamknúť ani odomknúť.

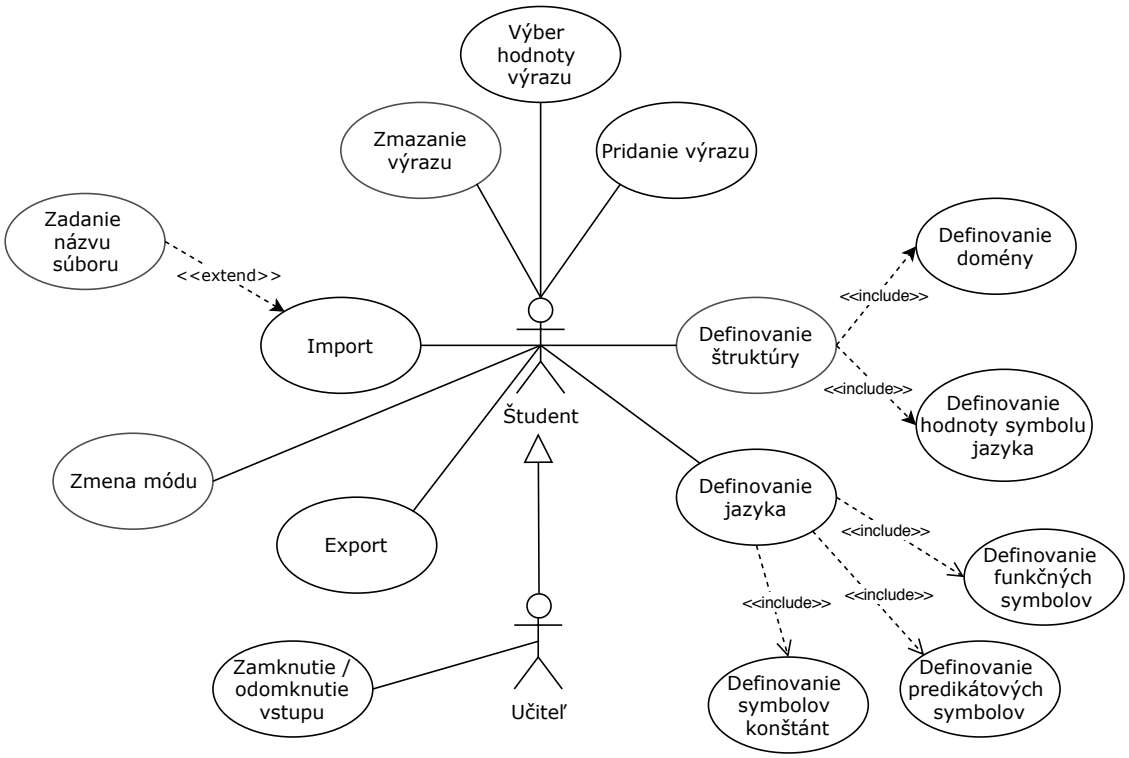
6) Uloženie a načítanie cvičenia

   a) Používateľ si bude môcť uložiť rozpracované cvičenie.

   b) Používateľ si bude môcť načítať rozpracované cvičenie.

2.3 Diagram prípadov použitia

Na obr. 2.1 je diagram prípadov použitia (use-case diagram) aplikácie. Zavádza dve používateľské roly – študent a učiteľ. Rola učiteľ dedí od roly študent a má k dispozícii jeden prípad použitia navyše – zamknutie a odomknutie vstupu.



Obr. 2.1: Diagram prípadov použitia aplikácie

2.4 Ostatné požiadavky

a) Aplikácia má byť naprogramovaná ako webová aplikácia na strane klienta.

b) Aplikácia má byť vysoko interaktívna a má okamžite reflektovať akúkoľvek zmenu.

c) Aplikácia nemusí mať implementovanú autentifikáciu užívateľa.

d) Bezpečný import.

2.5 Analýza aplikačnej domény

Aplikačnú doménu aplikácie tvorí logika prvého rádu, ktorú som opísal v stati 1.1.

Kapitola 3

Návrh

V tejto kapitole detailne popisujem návrh aplikácie. Popisujem návrh dátového modelu aplikácie, definujem gramatiky syntaxe textových vstupov a na záver uvádzam návrh grafického používateľského rozhrania aplikácie.

3.1 Dátový model

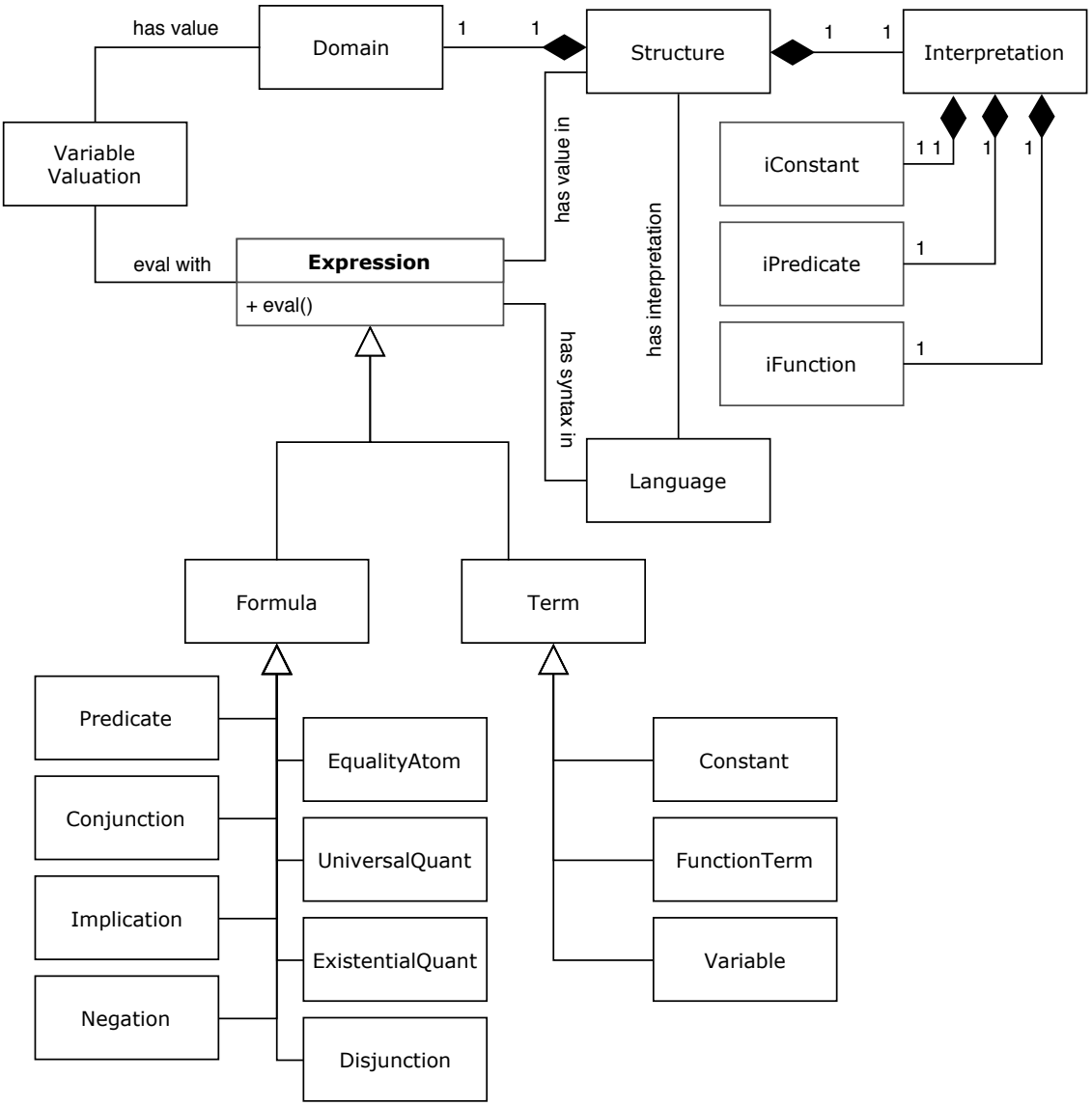
Na obr. 3.1 je zobrazený objektový návrh dátového modelu aplikácie.

Trieda `Language` bude ukladať symboly konštánt, predikátové symboly a funkčné symboly. Táto trieda je v asociácii s triedou `Expression`, pretože každý výraz musí obsahovať iba symboly nejakého jazyka. Taktiež je v asociácii s triedou `Structure`, kvôli interpretácii symbolov.

Trieda `Structure` si bude ukladať interpretácie symbolov jazyka – objekty `iConstant`, `iPredicate`, `iFunction`, a objekt domény `Domain`.

Objekt `VariableValuation` obsahuje ohodnotenie premenných a je osamostatnený od štruktúry. Je v asociácii s výrazmi a doménou štruktúry.

Trieda `Expression` je najzákladnejšia trieda výrazov. Výraz môže byť buď term alebo formula. Od triedy `Formula` dedia všetky typy formúl (predikátový a rovnostný atóm, formuly s unárnimi a binárnymi spojkami, kvantifikované formuly). Od triedy `Term` dedia zasa všetky typy termov (symbol konštanty, funkčný symbol a premenná). Každý výraz sa vyhodnocuje pomocou funkcie `eval()`. Výsledkom tejto funkcie v triede `Formula` je boolovská hodnota `true` alebo `false`, a v triede `Term` je to prvok domény štruktúry.



Obr. 3.1: Class diagram aplikácie

3.2 Návrh stavu aplikácie

Aplikácia bude obsahovať centrálny stav, ktorý budú tvoriť všetky dôležité dáta zadané používateľom, alebo vytvorené aplikáciou na základe týchto dát. Stav bude obsahovať nasledujúce informácie:

Štruktúra: V stave musí byť objekt štruktúry – trieda `Structure` (def. v stati 3.1).

Výrazy: Do stavu sa budú ukladať formuly a termy ktoré pridal používateľ. Každý výraz bude objekt, kde bude uložený samotný výraz (ktorý zadal používateľ), sparsovaná hodnota (inštancia triedy `Expression`), hodnota výrazu (`true` alebo `false`, resp. prvok domény) a spätná väzba (správa o tom, či má výraz správnu syntax)

Ohodnotenie premenných: Toto bude reprezentované ako štruktúra typu kľúč-hodnota na uchovanie ohodnotenia premenných.

Hodnoty textových vstupov: Tento objekt si bude ukladať informácie o každom textovom vstupe aplikácie, ako je informácia o tom, či je vstup zamknutý, spätná väzba pre používateľa (napr. či má zápis správnu syntax) a samotný text ktorý zadal používateľ.

Používateľský mód: Informácia o tom, v akom móde sa nachádza používateľ – v študentskom alebo učiteľskom. Predvolený mód bude študentský.

3.3 Syntax textových vstupov

V aplikácii sa bude vo veľkej miere syntakticky analyzovať text, ktorý zadá používateľ. Preto je potrebné jasne definovať syntaktické pravidlá (gramatiku) pre jednotlivé druhy vstupov. Gramatiky budem zavádzať v štandardnej Backus-Naurovej forme.

V nasledujúcich definíciách je veľakrát použitý neterminálny symbol `<spaces>`, čo je ľubovoľne dlhý reťazec pozostávajúci iba z medzier alebo tabulátorov. Ďalší veľakrát používaný symbol je `<identifier>`, ktorý prestavuje ľubovoľne dlhý reťazec.

3.3.1 Definovanie jazyka

Pri definovaní množiny symbolov jazyka platí jednotné pravidlo – jednotlivé symboly sú od seba oddelené čiarkou a môže byť medzi nimi ľubovoľný počet medzier. Vo výpise 3.1 je gramatika pre definovanie symbolov konštánt.

```
<constants> ::=
 <spaces> <identifier> (<spaces> "," <spaces> <identifier>)* <spaces>
```

Výpis 3.1: Gramatika definovania množiny symbolov konštánt

Gramatika definovania množiny predikátových alebo funkčných symbolov je podobná ako pri definovaní konštánt s tým rozdielom, že je potrebné uviesť aj aritu. Výpis 3.2 ukazuje túto gramatiku. Neterminálny symbol `<arity>` predstavuje celé kladné číslo.

```
<predicate>  ::= <spaces> <identifier> "/" <arity> <spaces>
<function>   ::= <spaces> <identifier> "/" <arity> <spaces>
<predicates> ::= <predicate> (<spaces> "," <predicate>)*
<functions>  ::= <function> (<spaces> "," <function>)*
```

Výpis 3.2: Gramatika definovania množiny predikátových a funkčných symbolov

3.3.2 Definovanie štruktúry

Vo výpise 3.3 je gramatika definovania množiny domény štruktúry. Neterminálny symbol `<domain_iden>` predstavuje identifikátor prvku domény – akýkoľvek reťazec unikódových znakov okrem medzier, čiarok a zátvoriek.

```
<domain> ::=
 <spaces> <domain_iden> (<spaces> "," <spaces> <domain_iden>)* <spaces>
```

Výpis 3.3: Gramatika definovania množiny domény štruktúry

Gramatika definovania interpretácie predikátového alebo funkčného symbolu cez textový vstup je vo výpise 3.4. Interpretácia sa bude definovať n-ticami pre n-árny predikátový symbol a $(n+1)$-ticami pre n-árny funkčný symbol. Počet členov n-tíc sa bude kontrolovať až po syntaktickej analýze.

```
<tuple> ::=
  "(" <spaces> <domain_iden> (<spaces> "," <domain_iden>)+ <spaces> ")"
 | <spaces> <domain_iden> <spaces>
<tuples> ::= <spaces> <tuple> (<spaces> "," <tuple>)* <spaces>
```

Výpis 3.4: Gramatika definovania interpretácie predikátových a funkčných symbolov

Gramatika ohodnotenia indivíduových premenných je vo výpise 3.5. Je veľmi podobná gramatike definovania interpretácie predikátových symbolov s tým rozdielom, že sú povolené iba dvojice, kde prvý prvok je názov premennej a druhý je prvok domény.

```
<e_tuple> ::=
 "(" <spaces><identifier><spaces> "," <spaces><domain_iden><spaces> ")"
<e_tuples> ::=
  <spaces> <e_tuple> (<spaces> "," <spaces> <e_tuple>)* <spaces>
```

Výpis 3.5: Gramatika definovania ohodnotenia indivíduových premenných

3.3.3 Výrazy

Gramatika výrazov (formuly a termy) sa bude držať pravidiel, ktoré som uviedol v stati 1.1.2. Vo výpise 3.6 je navrhnutá gramatika výrazov. Logické spojky sa budú dať zapísať unikódovými znakmi a zvyčajnými symbolmi z programovacích jazykov. Všimnite si, že logické symboly sa budú môcť zapisovať aj pomocou LaTeX príkazov. Preto je potrebné zabezpečiť, aby za takýmto príkazom bola minimálne jedna medzera. To reprezentuje symbol `<r_space>`. Vo všeobecnosti platí, že medzi symbolmi vo výraze môže byť ľubovoľný počet medzier.

Neterminálny symbol `<formula>` definuje gramatiku formuly. Formula môže byť tvorená binárnou spojkou, kde na pravej a ľavej strane spojky musí byť ďalšia podformula. Formula ďalej môže byť rovnostný alebo predikátový atóm, existenčný alebo všeobecný

kvantifikátor alebo negácia. Nakoniec, formula môže byť ľubovoľný početkrát vnorená do zátvoriek. V tom sa táto syntax mierne odlišuje od definície v stati 1.1.2

Neterminálny symbol `<term>` definuje gramatiku termu. Term môže byť symbol konštanty alebo symbol premennej. Gramatika funkčného termu je identická s gramatikou predikátového atómu a gramatika symbolu konštanty je identická so symbolom premennej.

Zvyšné neterminálne symboly definujú, aké symboly sa môžu použiť pri spojkách alebo kvantifikátoroch.

```
<formula> ::=
    "(" <formula> <spaces> <conjunction> <spaces> <formula> ")"
  | "(" <formula> <spaces> <disjunction> <spaces> <formula> ")"
  | "(" <formula> <spaces> <implication> <spaces> <formula> ")"
  | <term> <spaces> <equality> <spaces> <term>
  | <term> <spaces> <non_equality> <spaces> <term>
  | <exists> <spaces> <variable> <spaces> <formula>
  | <uni> <spaces> <variable> <spaces> <formula>
  | <negation> <formula>
  | <predicate>
  | "(" <formula> ")"

<conjunction> ::=
  "\wedge" <r_space> | "\land" <r_space> | "&&" | "&" | "∧" | "/\"
<disjunction> ::=
  "\vee" <r_space> | "\lor" <r_space> | "||" | "|" | "∨" | "\/"
<implication> ::= "\to" <r_space> | "→" | "->"

<exists> ::=
  "\exists" <r_space> | "\e" <r_space> | "\E" <r_space> | "∃"
<uni> ::=
  "\forall" <r_space> | "\a" <r_space> | "\A" <r_space> | "∀"

<negation> ::=
  "\neg" <r_space> | "\lnot" <r_space> | "-" | "!" | "~" | "¬"

<equality>     ::= "="
<non_equality> ::= "!=" | "<>" | "/=" | "≠"
<predicate>    ::= <identifier> <spaces> "(" <terms> ")"
<function>     ::= <predicate>
<term>         ::= <function> | <constant> | <variable>
<terms>        ::= <term> ("," <spaces> <term>)*
```

Výpis 3.6: Gramatika výrazu

3.4 Návrh komponentov používateľského rozhrania

Na obr. 3.2 je znázornené grafické rozloženie nasledujúcich piatich základných komponentov, z ktorých sa bude skladať používateľské rozhranie aplikácie:

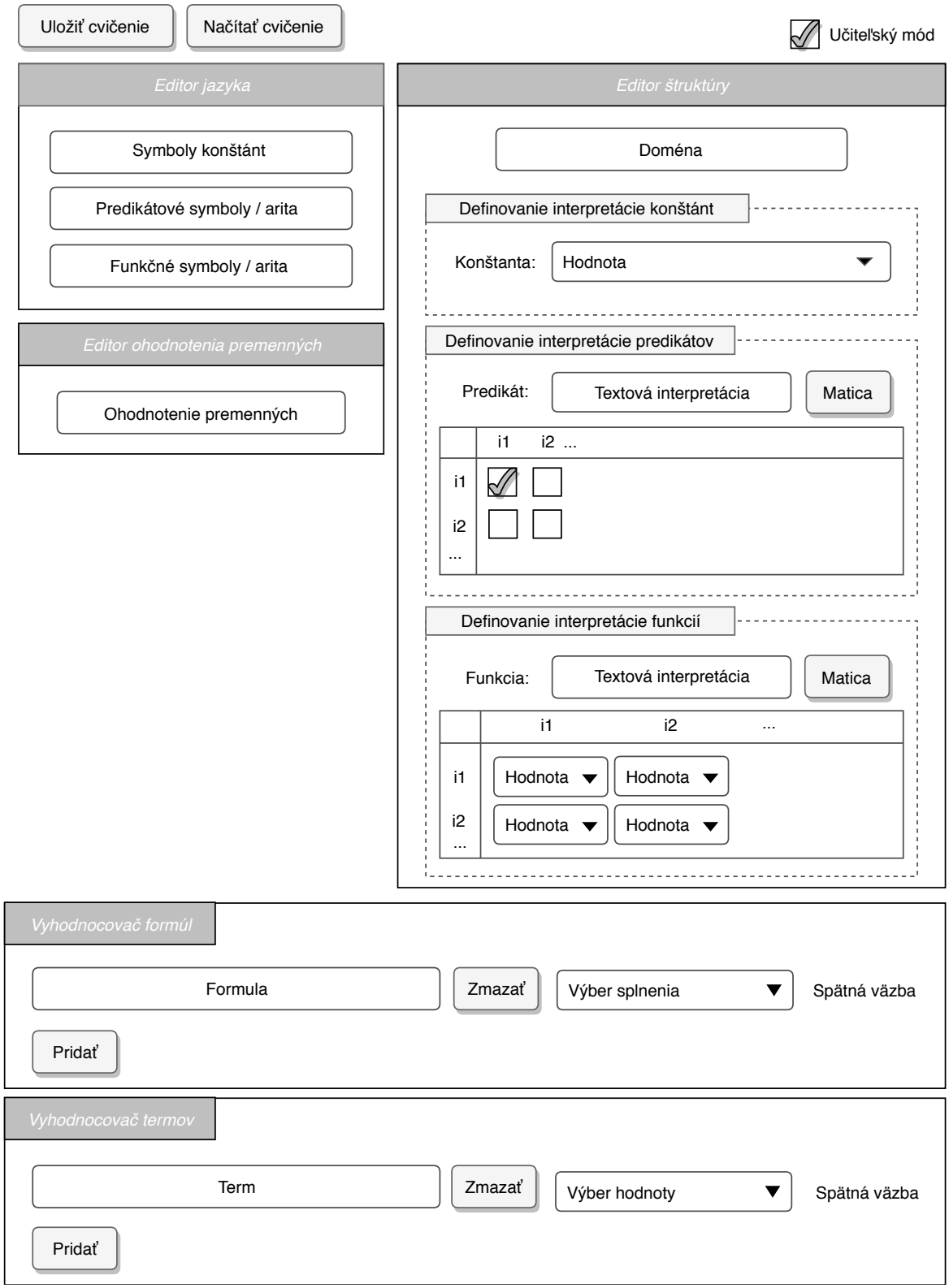
Editor jazyka: V komponente editore jazyka sa budú definovať mimologické symboly jazyka, ako sú symboly konštánt, funkčné symboly a predikátové symboly. Bude tvorený tromi textovými vstupmi na definovanie množín týchto symbolov.

Editor štruktúry: V komponente editora štruktúry sa bude definovať doména a interpretácie mimologických symbolov jazyka. Preto sa tento komponent bude skladať z troch podčastí – interpretácia symbolov konštánt, interpretácia predikátových symbolov a interpretácia funkčných symbolov. Podobne ako editor jazyka, aj tento sa bude skladať z textových vstupov. Interpretácia symbolu konštanty sa bude vyberať z menu, v ktorom budú prvky domény. Definovanie interpretácie predikátových a funkčných symbolov sa bude dať okrem textového vstupu aj pomocou editačnej matice. Matica bude tabuľka, v ktorej bunkách bude zaškrtávacie políčko pri predikátoch, resp. menu s prvkami domény pri funkciách.

Editor ohodnotenia premenných: Komponent ohodnotenia premenných bude slúžiť na definovanie ohodnotenia premenných. Komponent musí byť oddelený od editoru štruktúry, lebo ohodnotenie premenných nepatrí do štruktúry. Bude tvorený iba jedným textovým vstupom na zadanie interpretácie.

Vyhodnocovač formúl: Vo vyhodnocovači formúl si používateľ bude môcť vyhodnocovať, resp. kontrolovať formuly. Bude tvorený viacerými rovnakými podkomponentami, ktoré sa budú skladať z textového vstupu na zadanie formuly, menu na výber odpovede (či formula vyplýva zo štruktúry alebo nie) a spätnou väzbou, čo bude text o tom, či užívateľ vybral správnu odpoveď. Pri každej formule bude tlačidlo na jej okamžité zmazanie. Na konci bude tlačidlo na pridanie nového políčka pre zadanie a vyhodnotenie formuly.

Vyhodnocovač termov: Vyhodnocovač termov bude veľmi podobný vyhodocovaču formúl s tým rozdielom, že namiesto formúl sa tu budú vyhodnocovať iba termy a bude sa skladať z rovnakých častí. Vo výbere odpovede namiesto možnosti splnenia / nesplnenia tu budú prvky domény štruktúry. Rovnako ako pri formulách, aj tu bude tlačidlo na pridanie a zmazanie termu.



Obr. 3.2: Návrh grafického rozloženia komponentov aplikácie

3.5 Import a export

Definovaný jazyk, štruktúru a výrazy sa budú dať uložiť (exportovať) do počítača používateľa, a naspäť importovať. Toto bude realizované takým spúsobom, že sa bude dať exportovať stav aplikácie.

V stati 3.2 som navrhol stav aplikácie a čo všetko bude obsahovať. Bude obsahovať hodnoty textových vstupov hlavne kvôli tomu, aby sa z týchto hodnôt inicializoval objekt štruktúry `Structure`. Tento objekt preto nebude potrebné zahrnúť do exportovaných dát. Teda, v exportovaných dátach bude skoro celý stav (okrem objektu štruktúry).

Kapitola 4

Implementácia

V tejto kapitole sa venujem popisu implementácie návrhu aplikácie. Na začiatku uvádzam, aký implementačný jazyk som na implementáciu zvolil. Čitateľ sa tu oboznámi so štruktúrou aplikácie a významom jednotlivých priečinkov. Detailne opisujem stav, s ktorým aplikácia pracuje. Opisujem logiku syntaktického analyzátora a podávam jednoduchý manuál pre ďalší vývoj. Na záver uvádzam, ako sa aplikácia nainštaluje a URL adresu aplikácie.

4.1 Implementačný jazyk

Keďže aplikácia má byť vykonávaná na strane klienta a veľký dôraz sa kladie na rýchle a navzájom synchronizované užívateľské prostredie, rozhodli sme sa pre programovací jazyk JavaScript a jeho knižnicu React (opísaná je v stati 1.2.2). Využijem ju na vytvorenie rôznych prvkov grafického používateľského rozhrania a ochytávanie udalostí.

React je robustná knižnica na tvorbu používateľských rozhraní avšak jeho veľkou nevýhodou je, že nemá implementovaný centralizovaný manažment stavu aplikácie. V našej aplikácii sa bude narábať vo veľkej miere práve so stavom, kde sa budú ukladať všetky dôležité dáta. Preto sme sa rozhodli využiť ďalšiu JavaScriptovú knižnicu Redux (opísaná je v stati 1.2.3). Redux je knižnica, ktorá sa stará o manažment stavu aplikácie.

Na syntaktické analyzovanie textu, ktorý zadá používateľ, využijem syntaktický analyzátor PEG.js (opísaný je v stati 1.2.4), kvôli jednoduchosti a relatívne jednoduchej syntaxi zapisovania gramatiky.

4.2 Štruktúra aplikácie

V priečinku `/public/` sú verejné súbory ako sú skompilované CSS štýly a obrázky. Súbor `/public/index.html` je vstupný, obsahuje nalinkované dôležité súbory a element `<div id='root'></div>`, do ktorého React vloží komponenty aplikácie. Všetky zdrojové kódy

aplikácie sú v priečinku `/src/`, ktorého štruktúra je nasledovná:

- `actions/` – tu sú Reduxové akcie,
- `components/` – tu sú Reactové komponenty ktoré tvoria používateľské rozhranie,
- `constants/` – priečinok na ukladanie globálnych konštánt,
- `containers/` – tu sú Reduxové kontajnery,
- `model/` – tento priečinok obsahuje zdrojové súbory dátového modelu aplikácie opísaného v stati 3.1,
- `parser/` – priečinok vyhradený pre syntaktický analyzátor PEG.js,
- `reducers/` – tu sú uložené Reduxové reducery,
- `sass/` – štýly aplikácie v jazyku Sass, ktoré sú kompilované pomocou kompilátora Gulp do súboru `/public/css/style.css`,
- `tests/` – testy syntaktického analyzátora, reducerov a dátového modelu.

4.3 Štruktúra stavu

Vo výpise 4.1 je štruktúra centrálneho stavu `state`, s ktorým manipuluje aplikácia prostredníctvom Reduxu – modifikuje ho vykonávaním Reduxových akcií (opísané v stati 4.6), ktoré realizujú reducery (opísané v stati 4.7).

Atribút `structureObject` je objekt štruktúry (opísaný v stati 3.1). V atribúte `common` sa ukladajú všeobecné dáta, s ktorými aplikácia pracuje na rôznych miestach. Obsahuje `teacherMode`, učiteľský alebo študentský mód. Atribút `language` obsahuje dáta textových vstupov editora jazyka.

Atribút `structure` obsahuje dáta editora štruktúry. Atribúty `constants`, `predicates` a `functions` sú objekty, ktorých kľúče sú `názovSymbolu/aritaSymbolu`, a hodnotou sú dáta týkajúce sa toho konkrétneho symbolu. Pri predikátoch a funkciách je medzi dátami aj atribút `tableEnabled`, podľa ktorého je pri vstupe otvorená editačná matica alebo nie. Ďalej sa tu ukladajú dáta o vstupe editovania premenných a domény. Dáta ohodnotenia premenných obsahujú atribút `object`, kde sa ukladajú samotné hodnoty premenných.

Atribút `expressions` obsahuje všetky výrazy, ktoré pridal používateľ. Obsahuje dve polia `formulas` (formuly) a `terms` (termy). Každý výraz je reprezentovaný objektom, ktorý vracia funkcia `defaultExpressionData()`, kde `value` je výraz ktorý napísal používateľ, `expressionValue` je hodnota výrazu, `answerValue` je hodnota odpovede používateľa,

`errorMessage` je chybová správa, `inputLocked` a `answerLocked` je informácia, či je vstup na zadanie výrazu alebo výber odpovede zamknutý.

Funkcia `defaultInputData()` vracia predvolený objekt, ktorý nesie údaje o každom textovom vstupe. Do atribútu `value` sa ukladá text, ktorý zadá používateľ. Atribút `locked` informuje o tom, či je vstup zamknutý alebo nie, `errorMessage` je chybová hláška týkajúca sa vstupu (napr. z parsera) a `parsed` je výstup zo syntaktického analyzátora.

```
const state = {
  structureObject: new Structure(new Language()),
  common: { teacherMode: false },
  language: {
    constants: defaultInputData(),
    predicates: defaultInputData(),
    functions: defaultInputData(),
  },
  structure: {
    constants: {
      constant1/arity: {
        ...defaultInputData(), errorMessage: EMPTY_CONSTANT_VALUE
      }
    },
    predicates: {
      predicate1/arity: { ...defaultInputData(), tableEnabled: false }
    },
    functions: {
      function1/arity: { ...defaultInputData(), tableEnabled: false }
    },
    variables: { ...defaultInputData(), object: new Map() },
    domain: { ...defaultInputData(), errorMessage: EMPTY_DOMAIN },
  },
  expressions: {
   formulas: [defaultExpressionData(), ...],
   terms: [defaultExpressionData(), ...]
  }
};
const defaultInputData = () => ({
  value: '',
  locked: false,
  errorMessage: ''
  parsed: null
});
const defaultExpressionData = () => ({
  value: '',
  expressionValue: null,
  answerValue: '',
  errorMessage: '',
```

```
  inputLocked: false,
  answerLocked: false
});
```

Výpis 4.1: Štruktúra stavu aplikácie

4.4 Komponenty

Všetky komponenty sú v priečinku `components/`. Vstupný komponent je `App.js`, kde je inicializovaný `store` a funkcie na importovanie a exportovanie stavu. Tento komponent renderuje ostatné komponenty (resp. kontajnery komponentov) zaobalením ich do komponentu `<Provider store={store}>`. Takto sa ostatné komponenty vedia dostať k stavu aplikácie.

Kontajner `LanguageContainer` zaobaľuje komponent editoru jazyka `Language`. Tento je tvorený troma podkomponentami `TextInput`, čo sú textové vstupy na zadanie symbolov jazyka. Kontajner `StructureContainer` zaobaľuje komponent editoru štruktúry `Structure`.

4.5 Syntaktická analýza vstupov

Na syntaktickú analýzu vstupov využívam generátor syntaktických analyzátorov PEG.js, ktorého vstup a výstup sa nachádza v priečinku `/parser/`. Súbor `grammar.pegjs` obsahuje definície gramatík (ktoré sú navrhnuté a popísané v stati 3.3) a súbor `grammar.js` je vygenerovaný JavaScriptový kód z tejto gramatiky. Analyzátor sa generuje konzolovým príkazom napísaným vo výpise 4.2.

```
pegjs --allowed-start-rules formula,term,constants,predicates,functions,
    domain,tuples,e_tuples grammar.pegjs
```

Výpis 4.2: Generovanie syntaktického analyzátora

Analyzátor má 8 štartovacích neterminálnych symbolov: `formula`, `term`, `constants`, `predicates`, `functions`, `domain`, `tuples` a `e_tuples`. Ich význam bol opísaný v stati 3.3.

Výstupom analyzátora pre neterminály `formula` a `term` sú objekty príslušnej podtriedy triedy `Expression`. Pri ostatných neterminálochh je výstupom pole reťazcov, resp. polí reťazcov.

Funkcia `parseText(text, textData, parserOptions)` v priečinku `/reducers/functions/` dostáva ako parameter text na syntaktickú analýzu, objekt, kde sú uložené dáta o textovom vstupe (opísaný je v stati 4.3) a nastavenia analyzátora. Táto funkcia priamo mení `textData.parsed` kde uloží výstup z analyzátora, poprípade zmení

aj `textData.errorMessage`, ak by nastala nejaká chyba. Ak sa vstup nedá analyzovať, tak `textData.parsed` nastaví na `null`.

4.6 Akcie

Akcie, ktorých význam som popísal v stati 1.2.3, sú uložené v súbore `/actions/index.js`. Dali by sa zoskupiť do nasledovných skupín:

Akcie editoru jazyka: Do tejto skupiny patria akcie `setConstants`, `setPredicates` a `setFunctions`. Volajú sa pri zmene symbolov jazyka, tj. pri zmene hodnoty textového vstupu editoru jazkyka. Všetky tri sú úplne podobné a prijímajú iba text, ktorý napísal používateľ do textového vstupu.

Akcie editoru štruktúry: Tieto akcie sa zavolajú buď pri zmene domény štruktúry (`setDomain`), zmene ohodnotenia premenných (`setVariablesValue`), alebo zmene ohodnotenia nejakého symbolu jazyka. Pri zmene ohodnotenia symbolov konštánt je volaná akcia `setConstantValue`. Ak sa zmení ohodnotenie predikátového symbolu cez textový vstup, je volaná `setPredicateValueText`, a ak cez editačnú maticu, je volaná `setPredicateValueTable`. Podobne to platí aj pri zmene ohodnotenia funkčných symbolov. Akcia `toggleTable` je zavolaná pri kliknutí na tlačidlo zapnutia, resp. vypnutia editačnej matice pri ohodnotení predikátových a funkčných symbolov.

Akcie vyhodnocovačov výrazov: Tieto akcie zabezpečujú fungovanie vyhodnocovačov výrazov. Akcie `addExpression` a `removeExpression` sú volané pri pridávaní, resp. mazaní výrazu. Akcia `checkExpressionSyntax` dostáva text výrazu, ktorý napísal používateľ a pomocou nej sa skontroluje, či má správnu syntax. Akcia `setExpressionAnswer` je volaná pri výbere odpovede hodnoty, resp. pravdivostného ohodnotenia výrazu. Každá akcia z tejto skupiny dostáva argument `expressionType`, čo je typ výrazu – formula alebo term.

Zamykacie akcie: Tieto akcie s názvom začínajúcim na `lock` sa zavolajú pri kliknutí na tlačidlo zamknutia, resp. odomknutia nejakého textového vstupu.

Všeobecné akcie: Tieto akcie majú na starosti všeobecné úkony aplikácie. Patria sem `toggleTeacherMode` (zmena používateľského módu) a `importAppState` (načítanie stavu aplikácie).

4.7 Reducery a synchronizácia komponentov

Reducery sú funkcie, ktoré príjimajú ako argumenty akciu a aktuálny stav. Vrátia zmenenú kópiu stavu podľa toho, aká prišla akcia. Bolo možné napísať iba jeden reducer pre všetky akcie, ale kvôli prehľadnosti som ich rozštruktúroval do piatich. Všetky sú v priečinku `reducers/`.

Základný reducer `root.js` je vstupný reducer, ktorý volá ostatné, a ich výsledky spája do jedného objektu (nového stavu), ktorý vráti. V tomto reduceri sa definuje prvotný stav aplikácie. Reducer `language.js` má na starosti spracovanie jazyka. Reducer `structure.js` je obsiahlejší, a spracováva akcie ktoré sa týkajú nastavovania hodnôt v štruktúre alebo definovanie domény. Reducer `expressions.js` spracuváva akcie týkajúce sa výrazov - pridanie, zmazanie, vyhodnotenie, skontrolovanie syntaxe.

Synchronizovanosť aplikácie som vyriešil tak, že jednu akciu môžu obslúžiť viaceré reducery. Preto tu opíšem, ako reducery spracovávajú jednotlivé skupiny akcií definovaných v stati 4.6.

Ak sa zmení jazyk, musí sa skontrolovať štruktúra a aj výrazy. Preto akcie editoru jazyka spracovávajú všetky reducery. Napr. akciu `setConstants` spracuje reducer `language` kvôli zmene symbolov konštánt v objekte jazyka. Ďalej ju spracuje reducer `structure` kvôli pridaniu (resp. zmazaniu) vstupov pre zadanie hodnoty symbolov konštánt. Nakoniec akciu spracuje reducer `expressions`, aby sa výrazy znova syntakticky analyzovali a vyhodnotili (tak sa skontroluje, či je výraz stále syntakticky správny pre jazyk). Toto platí aj pre zvyšok akcií patriacich do tejto skupiny.

Pri zmene interpretácie symbolu jazyka, domény alebo ohodnotenia premenných stačí iba znova vyhodnotiť výrazy. Preto akcie editoru štruktúry spracuje reducer `structure` a aj `expressions`. Navyše, po zmene domény sa musia skontrolovať aj hodnoty všetkých symbolov jazyka (aby nemali hodnotu prvku, ktorý nie je v doméne).

Akcie vyhodnocovačov výrazov neovplyvňujú ostatné komponenty aplikácie. Preto tieto akcie spracováva iba reducer `expressions`.

4.8 Import a export

Stav aplikácie (popísaný v stati 4.3) sa dá exportovať do JSON súboru, ktorý sa uloží do počítača používateľa. Tento súbor sa dá následne importovať a tak inicializovať aplikáciu.

Funkcia `exportState()` v súbore `components/App.js` zabezpečuje exportovanie stavu aplikácie. Funkcia zoberie aktuálny stav a zmení ho na JSON formát. Nespracuje celý stav, ale vynechá `structureObject`, keďže tento objekt je možné inicializovať zo zvyšku stavu.

Funkcia `importState(e)` zabezpečuje importovanie stavu. Dostáva ako argument objekt elementu `<input type="file">` z ktorého prečíta obsah súboru a odošle akciu `importAppState` (popísanej v stati 4.6) do úložiska. Túto akciu najprv spracuje hlavný

reducer `root`, ktorý transformuje JSON formát do JavaScriptového objektu a v ňom vytvorí objekt štruktúry (`structureObject`). Týmto novým objektom prepíše aktuálny stav aplikácie. Akciu ďalej spracujú všetky ostatné reducery, aby sa všetky sparsované dáta vložili do objektu štruktúry.

4.9 Zdrojové kódy a nasadenie

Všetky zdrojové súbory sú dostupné

- v mojom repozitári na adrese `https://github.com/mcifra/prieskumnik-struktur`,
- alebo v repozitári organizácie FMFI-UK-1-AIN-412 na adrese `https://github.com/FMFI-UK-1-AIN-412/structure-explorer`.

Repozitár sa dá naklonovať nasledovne (repozitár organizácie FMFI-UK-1-AIN-412):

```
git clone https://github.com/FMFI-UK-1-AIN-412/structure-explorer.git
```

Výpis 4.3: Naklonovanie repozitára so zdrojovými súbormi aplikácie

Pre spustenie aplikácie lokálne je nutné nainštalovať potrebné balíčky, ktoré potrebuje aplikácia. Príkazy ako tieto balíčky nainštalovať a spustiť aplikáciu je vo výpise 4.4.

```
cd structure-explorer
npm install
npm start
```

Výpis 4.4: Nainštalovanie a spustenie aplikácie

Skompilovanú a minimalizovanú verziu aplikácie je možné nasadiť na vlastný webový server. Táto verzia je v priečinku `build/`. Pre úspešné skompilovanie a správne nalinkovanie súborov je potrebné zmeniť URL adresu, kde bude aplikácia v súbore `package.json` (atribút `homepage`). Príkaz pre skompilovanie je vo výpise 4.5.

```
npm run build
```

Výpis 4.5: Skompilovanie aplikácie

Aplikácia je nasadená na adrese
`https://fmfi-uk-1-ain-412.github.io/structure-explorer/`.

Kapitola 5

Testovanie

V tejto kapitole opisujem ako prebiehalo testovanie aplikácie a aké výsledky z neho vyplynuli.

5.1 Cieľ testovania

Cieľ testovania bol, aby sme zistili použiteľnosť aplikácie a to, či budú študenti vnímať nástroj ako užitočný. Ďalším dôvodom testovania bol, aby sme našli a opravili funkčné chyby aplikácie.

5.2 Priebeh testovania

Aplikáciu sme testovali ako beta verziu medzi študentami druhého, resp. tretieho ročníka na predmete Matematika (4) - Logika pre informatikov.

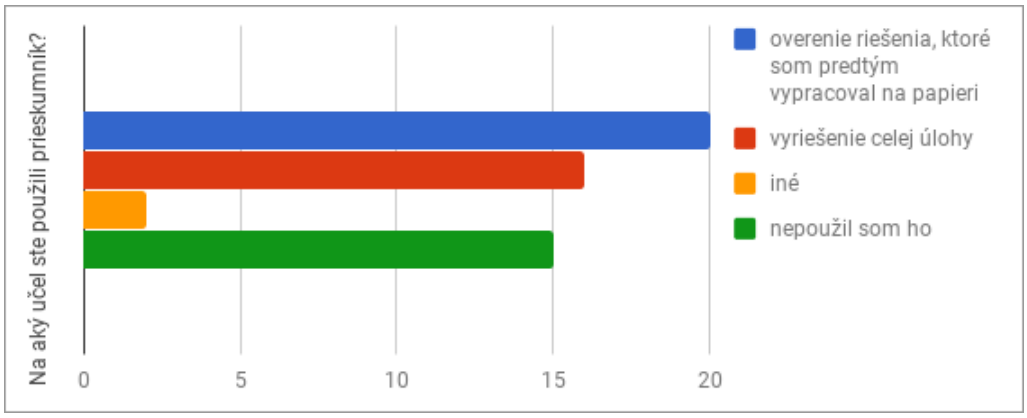
Testovanie prebiehalo nasledovne: študenti na riešenie dvoch domácich úloh (Príloha A) v týždňovom odstupe mali možnosť využiť náš nástroj. O týždeň neskôr sme študentov požiadali o vyplnenie dotazníka (Príloha B), kde sme sa ich pýtali na intuitívnosť používateľského rozhrania, na aký problém aplikáciu využili, akým spôsobom im nástroj pomohol a akú novú funkcionalitu by v nástroji privítali. Celkovo ho vyplnilo 47 študentov. Nižšie uvádzam výsledky tohto dotazníka.

5.3 Výsledky testovania

Funkčné chyby aplikácie Vďaka testovaniu sme zistili, aké funkčné chyby má aplikácia. Iba jeden študent našiel chybu, ktorá bola vďaka testovniu vyriešená. Táto chyba sa týkala definovania interpretácie funkčných symbolov v editore štruktúry. Bolo možné viackrát definovať interpretáciu funkčného symbolu, alebo ho nedefinovať pre všetky n-tice. Ďalšou chybou bolo, že pri vyhodnocovaní výrazu sa nekontrolovalo, či je definovaná

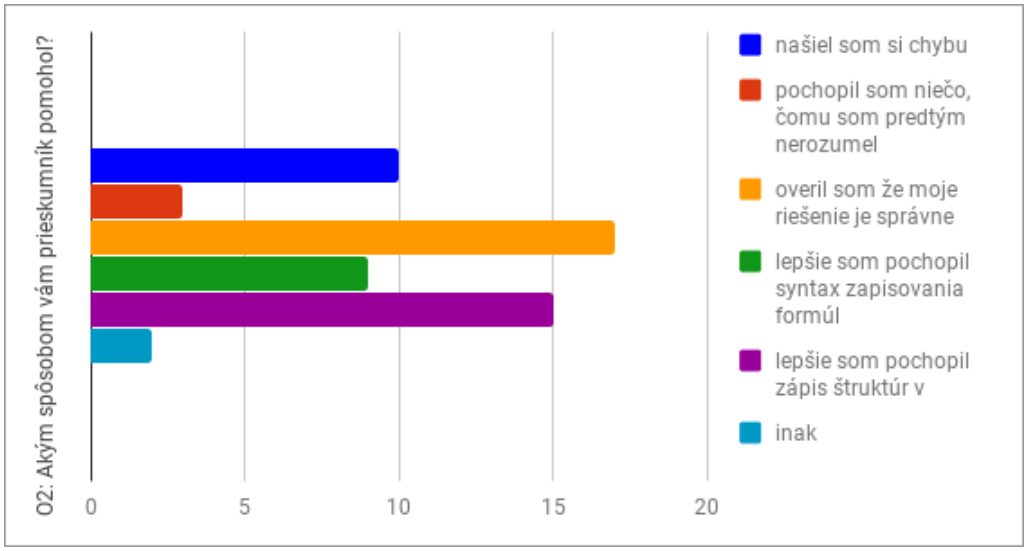
interpretácia nejakého mimologického symbolu jazyka (ktorý bol vo výraze). Dôsledkom tejto chyby boli niekedy chybné hodnoty vyhodnotenia výrazu.

Otázka 1 – Na aký účel ste použili prieskumník? Ako sa ukázalo, väčšina študentov nástroj využila kvôli overeniu riešenia. Zhruba jedna tretina študentov aplikáciu nevyužila. Najčastejšou príčinou bolo, že ju proste nepotrebovali. Predpokladá sa, že to boli študenti ktorým nerobila problém táto problematika pvrvorádovej logiky. Graf na obr. 5.1 znázorňuje výsledky tejto otázky.



Obr. 5.1: Na aký účel ste použili prieskumník? Výsledky sú v počte študentov.

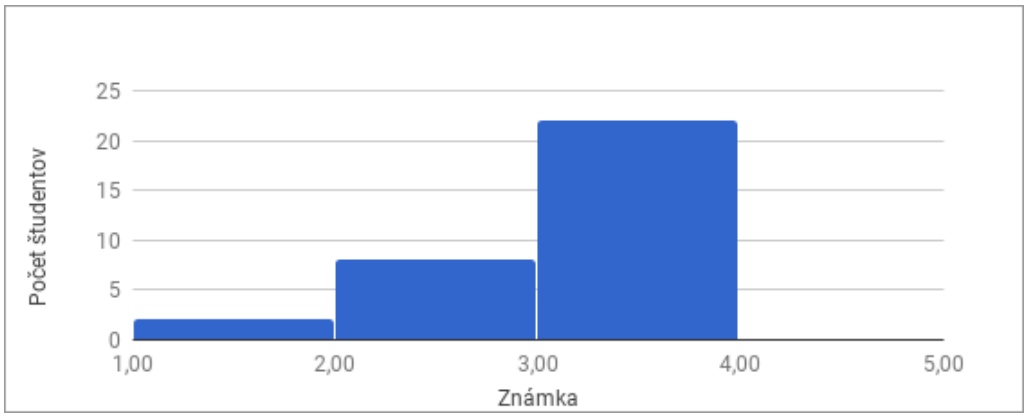
Otázka 2 – Akým spôsobom vám prieskumník pomohol? Z dotazníka ďalej vyplynulo, že aplikácia pomohla študentom pri overení riešenia, ktoré vypracovali na papier. Snahou a cieľom aplikácie bolo tiež striktne dodržiavať matematickú notáciu. Dotazník je dôkazom, že vďaka nemu študenti lepšie pochopili notáciu. Graf na obr. 5.2 znázorňuje výsledky tejto otázky.



Obr. 5.2: Akým spôsobom vám prieskumník pomohol? Výsledky sú v počte študentov.

Otázka 3 – Bolo používateľské rozhranie prieskumníka intuitívne? Priemer známok (čo znázorňuje histogram na obr. 5.3) intuitívnosti bol zhruba 3,74. To naznačuje, že prostredie je prehľadné, user-friendly ale má isté chyby. Medzi najčastejšie chyby používateľského rozhrania patrili:

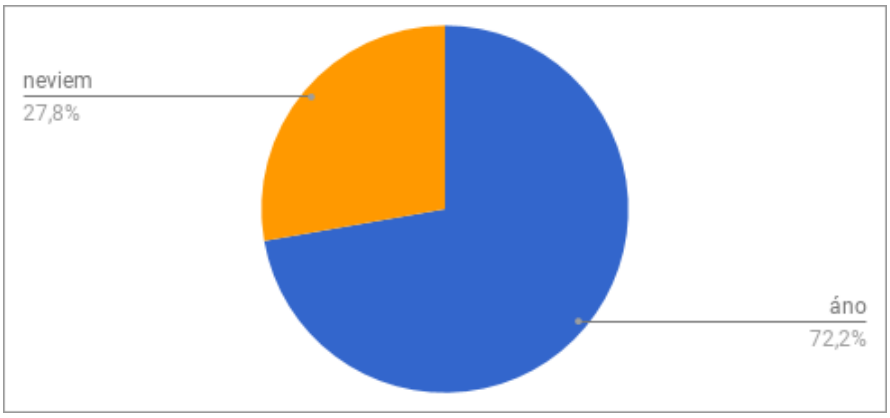
- chýbali príklady zápisu formúl,
- chýbali príkladové zápisy definovania štruktúry a jazyka,
- problémy zapisovania unikódových znakov vo formulách, chýbala virtuálna klávesnica, resp. tabuľka týchto znakov pre pohodlné kopírovanie,
- vstupy sú v niektorých prípadoch malé, pomohlo by ich ľubovoľne zväčšovať,
- popis chyby pri nejakom vstupe nebol dosť presný.



Obr. 5.3: Bolo používateľské rozhranie prieskumníka intuitívne?

Ak to zhrniem, najčastejšia kritika bola pri zápise formúl. Má to svoje opodstatnenie, keďže aplikácia v čase testovania mala len minimálny návod na používanie. Používateľom chýbali príklady, resp. návod a nevedeli ako zapisovať výrazy.

Otázka 4 – Odporučili by ste prieskumník ďalším študentom? Študenti by prevažne odporučili aplikáciu ďalším študentom. Niektorí nevedeli, pričom najčastejšou príčinou bol fakt, že aplikáciu využívali iba chvíľu alebo ju vôbec nepoužili. Tí ktorí by ju odporučili ako dôvod uviedli, že žiadna podobná neexistuje, resp. ju nepoznajú, je to dobrá pomôcka pri pochopení aj overení riešení a dá sa v nej štruktúra, resp. formuly ľahšie editovať ako na papieri. Nikto sa nenašiel, kto by aplikáciu neodporučil. Graf na obr. 5.4 znázorňuje výsledky tejto otázky.



Obr. 5.4: Odporučili by ste prieskumník ďalším študentom?

Otázka 5 – Akú novú funkcionalitu by ste privítali v prieskumníku? Návrhy na vylepšenie z piatej otázky sú nasledovné:

- pridať príklady zápisu výrazov do návodu,
- pridať virtuálnu klávesnicu na zadávanie špeciálnych unikódových znakov,
- pridať možnosť tlačiť vypracované cvičenie,
- napísať index v textovom reťazci, kde presne je chyba,
- pridať možnosť exportovať do formátu PDF, ktorý sa bude dať vylačiť a odovzdať.

Zhrnutie. Okrem zopár chýb ktoré boli odstránené sa ukázalo, že študentom sa aplikácia páčila, bola veľmi praktická a v niektorých prípadoch dokonca študentov naučila pochopiť štruktúry logiky prvého rádu. Na druhej strane, študenti sa sťažovali na problematické zapisovanie formúl, lebo nevedeli akým spôsobom ich majú zapisovať. Chýbal im návod, resp. príklad zápisu, ktorý bol neskôr pridaný. Ďalším problémom bolo, že nevedeli ako zapísať špeciálne unikódové znaky do výrazov. Tieto problémy sú námetom na ďalší vývoj aplikácie do takej podoby, kde budú odstránené všetky nedostatky ktoré sme zistili počas testovania.

5.4 Implementované vylepšenia

Medzi návrhmi a pripomienkami bolo pridanie lepšieho návodu, čo sme aj urobili. Pridali sme do návodu vyhodnocovača formúl zoznam spojok a kvantifikátorov ktoré sa dajú použiť. Takto sa jednotlivé znaky dajú skopírovať a čiastočne sa vyriešil aj návrh pridania virtuálnej klávesnice. Čiastočne sme vyriešili problém chýbajúcich príkladov zápisu výrazov alebo definovania jednotlivých množín. Pridali sme tieto príklady do editorov jazyka štruktúry a ohodnotenia premenných ako atribúty `placeholder` textových vstupov.

Vďaka testovaniu sme objavili jednu funkčnú chybu aplikácie ktorú sme úspešne odstránili. Teraz aplikácia upozorňuje používateľa o nejednoznačnosti, resp. neúplnosti interpretácie funkčného symbolu. Aplikácia nevyhodnotí výraz, ak v ňom nájde mimologický symbol jazyka ktorého interpretácia nie je definovaná v štruktúre.

Záver

Na záver môžem konštatovať, že sa mi podarilo splniť všetky ciele, ktoré boli požadované. Výsledkom práce je funkčná, stabilná a praktická aplikácia, ktorú študenti potrebovali. Aplikácia sa v určite bude v budúcnosti veľakrát používať. Pri testovaní sme dostali mnoho pozitívnych reakcií, no prišli aj pripomienky, ktoré sme z časti zapracovali. Aplikácia je vhodným kandidátom na ďalší vývoj v rámci ďalšej bakalárskej práce.

Vďaka vypracovaniu tejto práci som sa naučil mnoho nového. Mal som možnosť vytvoriť aplikáciu v modernej knižnici React a Redux, ktoré som predtým nepoznal. Naučil som sa zapisovanie gramatík a mal som možnosť testovať softvér v reálnych podmienkach.

Aplikáciu je možné ďalej rozvíjať v mnohých oblastiach. Jednou je vytvoriť grafické spracovanie definovania interpretácie symbolov jazyka. V tejto chvíli je možné interpretáciu definovať iba textovým vstupom alebo editačnou maticou. Ďalšou možnosťou pokračovania vývoja aplikácie je implementovať mechanizmus na napojenie prieskumníka do e-learningových systémov (napr. Moodle alebo courses.matfyz.sk). Aktuálne sa dá výstup aplikácie vytlačiť ako celá stránka (zo všetkými tlačidlami a vstupmi). Aby sa dali vytlačiť iba dôležité dáta, riešením by bolo CSS media type print.

Literatúra

[1] Dan Abramov and Andrew Clark. *Redux*. https://redux.js.org/. [Online, cit. 26.5.2018].

[2] Wikipedia contributors. React — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/React_(JavaScript_library), 2018. [Online; cit. 6.2.2018].

[3] Steve Faulkner, Aaron Eicholz, Travis Leithead, Alex Danilo, and Sangwhan Moon. *HTML 5.2*. World Wide Web Consortium, https://www.w3.org/TR/html52/, December 2017.

[4] Michael Genesereth and Eric Kao. Introduction to logic. http://logic.stanford.edu/intrologic/lessons/lessons.html. Prístupné dňa: 30.1.2018.

[5] Facebook Inc. *Hello world*. https://reactjs.org/docs/hello-world.html. [Online, cit. 11.2.2018].

[6] Ján Kľuka and Jozef Šiška. *Prednášky z Matematiky (4) – Logiky pre informatikov*. 2017. https://dai.fmph.uniba.sk/w/S%C3%BAbor:Course:Lpi-prednasky-2016-17.pdf.

[7] David Majda and Futago za Ryuu. *PEG.js*. https://pegjs.org/. [Online, cit. 11.2.2018].

[8] Rick Wong and Rolf van Widenfeit. Tarski's world. https://ggweb.gradegrinder.net/tarskisworld. Prístupné dňa: 30.5.2018.

[9] Štefan Ortutay. Logický pracovný stôl. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky, 2017.

[10] Monika Švaralová. Výukový program demonštrujúci matematický princíp. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyziky a informatiky, 2015.

[11] Vítězslav Švejdar. *Logika: neúplnost, složitost a nutnost*. Praha: Academia, 2002. http://www1.cuni.cz/~svejdar/book/LogikaSve2002.pdf.

Príloha A – Testovacie zadanie

Matematika 4 – Logika pre informatikov: Sada úloh 8

Riešenie teoretickej časti tejto sady úloh **odovzdajte** najneskôr v pondelok **23. apríla 2018 o 11:30** na prednáške.

Z tejto sady **budeme hodnotiť**: úlohu 2, praktickú úlohu 5 a jednu ďalšiu úlohu, ktorú vyžrebujeme na prednáške po termíne odovzdania.

Odovzdané riešenia musia byť **čitateľné** a mať primerane **malý** rozsah. Ohodnotené riešenia poskytneme k nahliadnutiu, ale **nevrátime** vám ich, uchovajte si kópiu. Na riešenia všetkých sád úloh sa vzťahujú všeobecné **pravidlá** zverejnené na oficiálnej stránke predmetu [`https://dai.fmph.uniba.sk/w/Course:Mathematics_4/sk#pravidla-uloh`].

Čísla úloh v zátvorkách pochádzajú zo zbierky, v ktorej nájdete ďalšie úlohy na precvičovanie a vzorové riešenia [`https://github.com/FMFI-UK-1-AIN-412/lpi/blob/master/ulohy/zbierka.pdf`].

Pri riešení niektorých úloh vám môže pomôcť prieskumník štruktúr [`http://www.st.fmph.uniba.sk/~cifra9/BP/#/app`].

Úloha 0 (nehodnotená). Zahrajte si formalizačný telefón: Posúvajte si lístky s tvrdeniami v slovenčine alebo formulami. Ak dostanete lístok s tvrdením v slovenčine, na nový čistý lístok ho sformalizujte. Ak dostanete lístok s formulou, zapíšte ju na nový čistý lístok čo najprirodzenejšou slovenčinou. Nový lístok posuňte ďalej.

Po aspoň troch opakovaniach porovnajte pôvodnú a konečnú verziu tvrdenia.

Úloha 1 (3.2.1). Uvažujme doménu rodinných vzťahov, ktorá už obsahuje predikáty ako žena^1, muž^1, rodič^2, súrodenec^2, kde $\text{žena}(x)$ znamená, že x je žena, $\text{muž}(x)$ znamená, že x je muž, $\text{rodič}(x, y)$ znamená, že x je rodičom y, $\text{súrodenec}(x, y)$ znamená, že x je súrodencom y. V prvorádovej logike napíšte definície nasledovných odvodených pojmov (tak, ako ich poznáte z prirodzeného jazyka):

(D_3) prastarý_rodič^2

(D_4) bratranec^2

(D_5) prasesternica^2 (teda sesternica „z druhého kolena“)

(D_6) $\text{nevlastný_súrodenec}^2$

(D_7) jedináčik^1

Úloha 2 (3.3.1, hodnotená). Zostrojte štruktúru $\mathcal{M} = (M, i)$ pre jazyk z predchádzajúcej úlohy tak, aby:

a) Štruktúra $\mathcal{M}$ splnila nasledujúce formuly v každom ohodnotení:

(A_1) $\exists x \exists y (\text{rodič}(x, \text{Andrea}) \wedge \text{rodič}(y, \text{Andrea}) \wedge \text{rodič}(x, \text{Boris}) \wedge \text{rodič}(y, \text{Boris}))$,

1

(A_2) $(\text{rodič}(\text{Andrea}, \text{Cyril}) \wedge \text{rodič}(\text{Boris}, \text{Diana}))$,

(A_3) $\forall x\, \neg\text{rodič}(x, x)$,

(A_4) $\forall x\big((\text{žena}(x) \vee \text{muž}(x)) \wedge \neg(\text{žena}(x) \wedge \text{muž}(x))\big)$,

(A_5) $\forall p \forall q \forall r \forall x\big((\text{rodič}(p, x) \wedge \text{rodič}(q, x) \wedge \text{žena}(p) \wedge \text{žena}(q)) \rightarrow p \doteq q\big)$,

(A_6) $\forall p \forall q \forall r \forall x\big((\text{rodič}(p, x) \wedge \text{rodič}(q, x) \wedge \text{muž}(p) \wedge \text{muž}(q)) \rightarrow p \doteq q\big)$,

(A_7) $\forall x \forall y\big(\text{súrodenec}(x, y) \leftrightarrow \big(\neg\, x \doteq y \wedge \exists z(\text{rodič}(z, x) \wedge \text{rodič}(z, y))\big)\big)$.

b) Štruktúra $\mathcal{M}$ *navyše* splnila nasledujúce formulu a definície v nich použitých pojmov z úlohy 1:

(B_1) $\exists x \exists y\, \text{prastarý_rodič}(x, y)$,

(B_2) $\exists x\big(\text{jedináčik}(x) \wedge \forall y(\text{rodič}(x, y) \rightarrow \text{jedináčik}(y))\big)$,

(B_3) $\exists x \exists y(\text{súrodenec}(x, y) \wedge \text{nevlastný_súrodenec}(x, y))$.

Prosíme vás, aby ste vo svojom riešení uviedli, či ste využili prieskumník štruktúr [`http://www.st.fmph.uniba.sk/~cifra9/BP/#/app`].

Úloha 3 (3.3.2). Dokážte, že nasledujúce tvrdenia *nie sú* ani platné ani nesplniteľné:

e) $\big((\neg\forall x\, P(x) \vee \forall x\, Q(x)) \rightarrow \forall x(P(x) \rightarrow Q(x))\big)$,

f) $\big((\exists x\, P(x) \wedge \exists x\, Q(x)) \rightarrow \exists x(P(x) \wedge Q(x))\big)$.

Prosíme vás, aby ste vo svojom riešení uviedli, či ste využili prieskumník štruktúr [`http://www.st.fmph.uniba.sk/~cifra9/BP/#/app`].

Úloha 4 (3.1.8). Vyjadrite čo najprirodzenejšími slovenskými vetami nasledujúcu formalizáciu zistení o deťoch a Vianociach v jazyku $\mathcal{L}$ s množinami symbolov $\mathcal{V}_{\mathcal{L}} = \{u, v, w, x, y, z\}$, $\mathcal{C}_{\mathcal{L}} = \{\text{Vianoce}, \text{Ježiško}, \text{Santa}, \text{Anička}\}$ a $\mathcal{P}_{\mathcal{L}} = \{\text{autíčko}^1, \text{blud}^1, \text{dieťa}^1, \text{dobrý}^1, \text{dostane}^2, \text{chlapec}^1, \text{kriticky_myslí}^1, \text{teší_sa_na}^2, \text{uhlie}^1, \text{verí_v}^2\}$:

(V_1) $\forall x\big(\big(\text{dieťa}(x) \wedge (\text{verí_v}(x, \text{Ježiško}) \vee \text{verí_v}(x, \text{Santa}))\big) \rightarrow \text{teší_sa_na}(x, \text{Vianoce})\big)$,

(V_2) $\forall x\big(\text{dieťa}(x) \wedge \neg\text{verí_v}(x, \text{Santa}) \rightarrow (\text{verí_v}(x, \text{Ježiško}) \vee \text{kriticky_myslí}(x))\big)$,

(V_3) $\forall x\big(\text{kriticky_myslí}(x) \rightarrow \forall y(\text{blud}(y) \rightarrow \neg\text{verí_v}(x, y))\big)$,

(V_4) $\forall x\big(\neg\text{dobrý}(x) \rightarrow \big(\neg\exists y\, \text{dostane}(x, y) \vee (\text{verí_v}(x, \text{Santa}) \rightarrow \exists y(\text{dostane}(x, y) \wedge \text{uhlie}(y)))\big)\big)$,

(V_5) $\forall x\big(\texttt{dobrý}(x) \wedge \texttt{chlapec}(x) \rightarrow \exists y(\texttt{dostane}(x,y) \wedge \texttt{autíčko}(y))\big)$.

Úloha 5 (praktická, odovzdávaná a hodnotená osobitne). Vyriešte a odovzdajte podľa pokynov praktické cvičenie `cv08`
`https://github.com/FMFI-UK-1-AIN-412/lpi/tree/master/cvicenia/cv08`.

Matematika 4 – Logika pre informatikov: Sada úloh 9

Riešenie teoretickej časti tejto sady úloh **odovzdajte** najneskôr v pondelok **30. apríla 2018 o 11:30** na prednáške.

Z tejto sady **budeme hodnotiť**: úlohu 3, praktickú úlohu 4 a jednu ďalšiu úlohu, ktorú vyžrebujeme na prednáške po termíne odovzdania.

Odovzdané riešenia musia byť **čitateľné** a mať primerane **malý** rozsah. Ohodnotené riešenia poskytneme k nahliadnutiu, ale **nevrátime** vám ich, uchovajte si kópiu. Na riešenia všetkých sád úloh sa vzťahujú všeobecné **pravidlá** zverejnené na oficiálnej stránke predmetu [`https://dai.fmph.uniba.sk/w/Course:Mathematics_4/sk#pravidla-uloh`].

Čísla úloh v zátvorkách pochádzajú zo zbierky, v ktorej nájdete ďalšie úlohy na precvičovanie a vzorové riešenia [`https://github.com/FMFI-UK-1-AIN-412/lpi/blob/master/ulohy/zbierka.pdf`].

Pri riešení niektorých úloh vám môže pomôcť prieskumník štruktúr [`http://www.st.fmph.uniba.sk/~cifra9/BP/#/app`].

Úloha 1 (3.4.1). Nájdite štruktúru, ktorá splní prvorádovú teóriu $T = \{A_1, \ldots, A_6\}$ v jazyku $\mathcal{L}$, kde $\mathcal{V}_\mathcal{L} = \{u, v, w, x, y, z\}$,
$C_\mathcal{L} = \{\text{Andrea}, \text{Danka}, \text{Hanka}, \text{Janka}, \text{Max}, \text{Nikita}\}$,
$\mathcal{F}_\mathcal{L} = \{\text{manžel}^1, \text{manželka}^1, \text{prvorodené_dieťa}^2\}$,
$\mathcal{P}_\mathcal{L} = \{\text{Lekár}^1, \text{Manželia}^2, \text{Muž}^1, \text{Právnik}^1, \text{Žena}^1\}$.

(A_1) $\forall x \forall y (\text{Manželia}(x, y) \to \text{Manželia}(y, x))$

(A_2) $\forall x \forall y \big(\text{Manželia}(x, y) \to$
$(\text{Muž}(x) \to x \doteq \text{manžel}(y)) \wedge (\text{Žena}(x) \to x \doteq \text{manželka}(y))\big)$

(A_3) $\forall x \forall y \big(\text{Manželia}(x, y) \wedge \text{Muž}(x) \wedge \text{Žena}(y) \to$
$\text{Lekár}(\text{prvorodené_dieťa}(x, y)) \vee \text{Právnik}(\text{prvorodené_dieťa}(x, y))\big)$

(A_4) $\text{Manželia}(\text{Hanka}, \text{Max}) \wedge \text{Žena}(\text{Hanka}) \wedge \text{Muž}(\text{Max})$

(A_5) $\text{Manželia}(\text{Danka}, \text{Janka}) \wedge \text{Žena}(\text{Danka}) \wedge \text{Žena}(\text{Janka})$

(A_6) $\text{Manželia}(\text{Andrea}, \text{Nikita}) \wedge \text{Muž}(\text{Andrea})$

Prosíme vás, aby ste vo svojom riešení uviedli, či ste využili prieskumník štruktúr [`http://www.st.fmph.uniba.sk/~cifra9/BP/#/app`].

Úloha 2 (3.5.1, 3.5.2). Pre každú z nasledujúcich formúl

i. vyznačte oblasti platnosti kvantifikátorov,

ii. určte množinu voľných premenných,

iii. zistite, či je substitúcia aplikovateľná,

iv. ak je substitúcia aplikovateľná, určte výsledok substitúcie.

a) $\big(P(y) \land \exists x\,(Q(x,y) \lor R(x)) \to S(x)\big)\{x \mapsto c, y \mapsto f(x)\}$

b) $\forall z\,\big(P(x,z) \land \exists w\,(R(w) \to Q(x, g(y,z)))\big)\{x \mapsto f(y), y \mapsto g(x,y), w \mapsto g(a)\}$

Úloha 3 (3.6.1, 3.6.2, 3.6.3, hodnotená). Dokážte tablovým kalkulom:

a) $\models \forall x(\texttt{hladná}(x) \land \neg\texttt{vyspatá}(x)) \leftrightarrow (\forall x\,\texttt{hladná}(x) \land \neg\,\exists x\,\texttt{vyspatá}(x))$,

b) $\{\forall x\big(\,P\big(g(x)\big) \to \exists y\,Q(x, g(y))\,\big)\} \models \exists x\big(P(x) \to \forall y\,\exists z\,Q(f(y), z)\big)$.

Úloha 4 (praktická, odovzdávaná a hodnotená osobitne). Vyriešte a odovzdajte podľa pokynov praktické cvičenie `cv09`
`https://github.com/FMFI-UK-1-AIN-412/lpi/tree/master/cvicenia/cv09`.

Príloha B - Dotazník

Prosím o vyplnenie krátkeho dotazníka, ktorý je súčasťou testovania webového nástroja a mojej bakalárskej práce „Prieskumník štruktúr“.

1. Na aký účel ste použili prieskumník? (označte viac možností)
   a. overenie riešenia, ktoré som predtým vypracoval(a) na papieri
   b. vyriešenie celej úlohy
   c. iné (napíšte)
      ……..
   d. nepoužil(a) som ho, pretože (napíšte)
      ……..
2. Akým spôsobom vám prieskumník pomohol? (označte viac možností)
   a. našiel/našla som si chybu
   b. pochopil(a) som niečo, čomu som predtým nerozumel(a) (napíšte, čo to bolo)
      ……..
   c. overil(a) som, že moje riešenie je správne
   d. lepšie som pochopil(a) syntax zapisovania formúl
   e. lepšie som pochopil(a) zápis štruktúr v matematickej notácii
   f. inak (napíšte ako)
      ……..
3. Bolo používateľské rozhranie prieskumníka intuitívne? (označte na stupnici)

1	2	3	4	5
bolo úplne neprehľadné				bolo úplne intuitívne

   Čo by sa mohlo na používateľskom rozhraní zmeniť, resp. čo robilo problémy?
   ……
   ……
4. Odporučili by ste prieskumník ďalším študentom? (označte jednu možnosť)
   a. áno (napíšte prečo) ……..
   b. nie (napíšte prečo) ……..
   c. neviem
5. Akú novú funkcionalitu by ste privítali v prieskumníku?
   ……..
   ……..

Ďakujem za váš čas a vyplnenie dotazníka! Milan Cifra (autor)

Príloha C - Elektronická príloha

Obsahom priloženého CD sú:

- v priečinku `src/` zdrojové súbory implementovanej aplikácie (návod na ich spustenie je v stati ...),
- v priečinku `build/` skompilovaná a minimalizovaná verzia, ktorú je možné priamo spustiť otvorením súboru `index.html` vo webovom prehliadači.

Zdrojové texty sú tiež dostupné na adrese `https://github.com/FMFI-UK-1-AIN-412/structure-explorer`.

Funkčná aplikácia je prístupná na adrese `https://fmfi-uk-1-ain-412.github.io/structure-explorer/`.